

**PEMBUATAN MIDDLEWARE KOMUNIKASI
MENGUNAKAN XMPP dan ARSITEKTUR
MICROSERVICE**

Skripsi

**Disusun untuk memenuhi salah satu syarat
Memperoleh gelar Sarjana Komputer**



Resa Fajar Sukma

1313618029

**Program Studi Ilmu Komputer
Fakultas Matematika dan Ilmu Pengetahuan Alam
Universitas Negeri Jakarta
2025**

LEMBAR PENGESAHAN

Dengan ini saya mahasiswa Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Jakarta

Nama : RESA FAJAR SUKMA
No. Registrasi : 1313618029
Jurusan : Ilmu Komputer
Judul : Pembuatan *Middleware* Komunikasi
Menggunakan XMPP dan Arsitektur
Microservice

Menyatakan bahwa skripsi ini telah siap diajukan.

Menyetujui,

Dosen Pembimbing I



Muhammad Eka Suryana,
M.Kom.

NIP. 19851223 201212 1 002

Dosen Pembimbing II

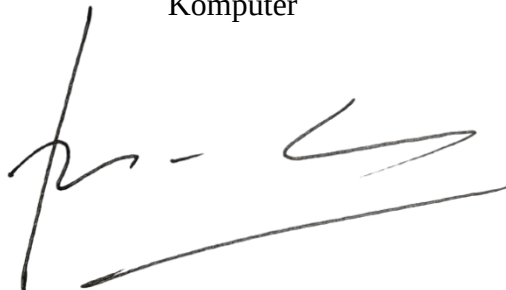


Med Irzal, M.Kom.

NIP. 19770615 200312 1 001

Mengetahui

Koordinator Program Studi Ilmu
Komputer



Dr. Ria Arafiah, M. Si.

NIP. 19751121 200501 2 004

KATA PENGANTAR

Puja dan puji syukur dipanjatkan ke hadirat ALLAH SWT, dengan rahmat dan karunia yang diberikan skripsi berjudul *Pembuatan Middleware Menggunakan XMPP dan Arsitektur Microservice* dapat terselesaikan.

Rasa terima kasih yang sangat besar disampaikan kepada pihak yang telah membantu jalannya penulisan skripsi ini dari awal hingga selesai. Kepada.

1. Kedua orang tua dan adik yang mendukung, mendoakan, serta bersabar selama pembuatan skripsi ini.
2. Dosen pembimbing penulis, yang membantu dalam skripsi ini, yth. Bapak Muhammad Eka Suryana, M.Kom. selaku dosen pembimbing I, dan yth. Bapak Med Irzal, M.Kom. selaku dosen pembimbing II.
3. Teman terdekat yang selalu mendukung, membantu, dan mendengarkan permasalahan yang terjadi.
4. Teman-teman mantan BEM ILKOM UNJ angkatan 2018 yang selalu bertanya dan memberikan dorongan.
5. Koordinator PRODI ILKOM UNJ, yth. Ibu Dr. Ria Arafiah, M.Si.

Skripsi ini masih jauh dari kata sempurna karena keterbatasan ilmu dan pengetahuan penulis. Kritik dan saran yang membangun akan disambut dengan senang hati oleh penulis. Skripsi ini diharapkan akan membantu dan memperluas pengetahuan semua pihak termasuk penulis sendiri. Semoga ALLAH SWT senantiasa membalas kebaikan semua pihak yang telah membantu dalam penulisan skripsi ini hingga selesai.

Jakarta, 13 Desember 2024

Resa Fajar Sukma

ABSTRAK

Resa Fajar Sukma. Pembuatan Middleware Komunikasi Menggunakan XMPP dan Arsitektur Microservice. Skripsi. Program Studi Ilmu Komputer. Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Negeri Jakarta. Januari 2025. Di bawah bimbingan Muhammad Eka Suryana, M. Kom. dan Med Irzal M. Kom.

Aplikasi membutuhkan waktu, tenaga, dan biaya untuk mengembangkannya. Salah satu fitur yang umum terdapat pada aplikasi adalah komunikasi. Penelitian ini dibuat untuk memangkas waktu pengembang aplikasi dalam mengembangkan fitur komunikasi, dengan cara menggunakan *middleware* yang sudah dapat digunakan. *Middleware* ini menggunakan XMPP sebagai protokol pertukaran data, XMPP menggunakan socket programming tipe *long-lived TCP connection*, dan *middleware* menggunakan arsitektur *microservice* untuk membagi setiap fungsionalitas besarnya. Fungsionalitas dibagi menjadi tiga bagian besar yaitu pengalamatan setiap komponen yang diwakili oleh komponen *tracker*, proses manajerial yang diwakili oleh komponen *manager*, dan pengiriman perpesanan yang diwakili oleh komponen *relay*. Tujuan utama dari penelitian ini adalah mengubah arsitektur monolith XMPP menjadi *microservice* yang akan memberikan kemampuan setiap fungsionalitas besar menjadi *best practice* terhadap kasus yang dihadapi oleh masing-masing fungsionalitas.

Hasil dari penelitian ini menunjukkan perubahan dari *monolith* ke *microservice*, memberikan keleluasaan dalam pengembangan tiap komponen dan penerapannya, tanpa takut merusak komponen lainnya serta memberikan hasil positif berupa kemampuan menerapkan *best practice* dari setiap kasus yang dihadapi komponen.

Kata kunci: *middleware, XMPP, socket programming, microservice*

ABSTRACT

Resa Fajar Sukma. Pembuatan Middleware Komunikasi Menggunakan XMPP dan Arsitektur Microservice. Mini Thesis. Computer Science. Faculty of Mathematics and Natural Sciences, State University of Jakarta. January 2025. Under the guidance of Muhammad Eka Suryana, M. Kom. and Med Irzal, M. Kom.

Developing an application requires time, effort, and resources. One common feature found in applications is communication. This research aims to reduce the time developers spend developing communication features by utilizing existing middleware. This middleware uses XMPP as the data exchange protocol, XMPP employs long-lived TCP connection socket programming, and the middleware utilizes a microservice architecture to divide large functionalities. Functionalities are divided into three main parts: addressing each component represented by the tracker component, the managerial process represented by the manager component, and message delivery represented by the relay component. The primary objective of this research is to transform the monolithic XMPP architecture into a microservice architecture, which will empower each major functionality to become a best practice for the specific cases faced by each functionality.

The results of this research demonstrate that the transformation from monolithic to microservice provides flexibility in developing and implementing each component without the fear of damaging other components, and yields positive results in the ability to apply best practices for each case faced by the component.

Keyword: *middleware, XMPP, socket programming, microservice*

DAFTAR ISI

LEMBAR PENGESAHAN	ii
KATA PENGANTAR.....	iii
ABSTRAK	iv
ABSTRACT	v
DAFTAR ISI.....	vi
DAFTAR GAMBAR.....	x
DAFTAR TABEL	xiv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah.....	2
1.4 Tujuan Penelitian	3
1.5 Manfaat Penelitian	3
BAB II KAJIAN PUSTAKA	4
2.1 Jaringan Komputer	4
2.2 <i>Socket</i>	4
2.2.1 <i>Socket</i> Protokol TCP	5
2.2.2 <i>Socket</i> Protokol UDP	5
2.3 <i>IP Address</i>	6
2.3.1 IPv4	6
2.3.2 IPv6	6
2.3.3 Alamat IP Publik	6
2.3.4 Alamat IP <i>Private</i>	7
2.3.5 Komunikasi Antar Alamat IP	7
2.4 <i>Relay</i>	8

2.5 <i>Middleware</i>	9
2.6 <i>Monolithic</i>	9
2.7 <i>Microservice</i>	10
2.8 VPS (<i>Virtual Private Server</i>)	10
2.9 Python 3	11
2.10 XMPP.....	12
2.10.1 <i>Services</i>	13
2.10.2 Aplikasi	14
2.10.3 Arsitektur XMPP.....	14
2.10.4 Pengalamatan XMPP	16
2.10.5 DNS (<i>Domain Name System</i>).....	17
2.10.6 <i>Resource</i>	17
2.10.7 Komunikasi XMPP	17
2.10.8 <i>Stanza Message</i>	18
2.10.9 <i>Stanza Presence</i>	19
2.10.10 <i>Stanza IQ (Info/Query)</i>	20
2.11 Perbandingan Mengirim Data dengan XML dan JSON	23
BAB III METODOLOGI PENELITIAN	26
3.1 Tahapan Penelitian	26
3.2 Arsitektur <i>Middleware</i> Komunikasi dengan XMPP	27
3.2.1 Komponen <i>Tracker</i>	28
3.2.2 Komponen <i>Manager</i>	28
3.2.3 Komponen <i>Relay</i>	29
3.2.4 Komponen <i>User</i>	30
3.2.5 Relasi <i>Tracker</i> dengan Komponen Lainnya.....	32
3.2.6 Relasi <i>Relay</i> dengan <i>Manager</i>	32

3.2.7 Relasi <i>User</i> dengan <i>Manager</i>	32
3.2.8 Relasi <i>User</i> dengan <i>Relay</i>	33
3.2.9 Relasi Antar <i>Relay</i>	33
3.3 Format XMPP	33
3.4 Penyimpanan Data	34
3.4.1 Struktur Penyimpanan Data <i>Tracker</i>	34
3.4.2 Struktur Penyimpanan Data <i>Manager</i>	35
3.4.3 Struktur Penyimpanan Data <i>Relay</i>	36
3.4.4 Struktur Penyimpanan Data <i>User</i>	36
3.5 Alat dan Bahan Penelitian	37
3.6 Tahapan Pengembangan	37
3.6.1 Pemilihan VPS	37
3.6.2 Pengembangan Komponen	37
3.7 Skema Uji	37
3.7.1 Metrik Pengujian	38
3.7.2 Rancangan Pengujian	38
BAB IV HASIL DAN PEMBAHASAN	39
4.1 Implementasi Lingkungan Program dan Sistem	39
4.2 Utilitas Umum Komponen	40
4.2.1 Kelas <i>SocketServer</i>	40
4.2.2 Utilitas <i>Get_Time</i>	41
4.3 Implementasi Komponen <i>Tracker</i>	41
4.3.1 Diagram Komponen <i>Tracker</i>	41
4.3.2 Cara Kerja <i>Tracker</i>	42
4.4 Implementasi Komponen <i>Manager</i>	44
4.4.1 Diagram Komponen <i>Manager</i>	44

4.4.2 Inisiasi Manager	45
4.4.3 Pengelolaan Autentikasi.....	47
4.4.4 Pengelolaan Jumlah Koneksi Pada Relay	50
4.4.5 Pengelolaan Roster.....	51
4.4.6 Pengelolaan Presence	54
4.5 Implementasi Komponen <i>Relay</i>	57
4.5.1 Diagram Komponen <i>Relay</i>	57
4.5.2 Inisiasi <i>Relay</i>	57
4.5.3 Pengelolaan <i>User</i> Antar <i>Relay</i>	60
4.5.4 Pengelolaan <i>Stanza Message User</i>	62
4.5.5 Pengelolaan <i>Stanza Message</i> Antar <i>Relay</i>	63
4.6 Implementasi Komponen Penguji atau <i>User</i>	64
4.7 Pengujian Proses Komunikasi.....	70
4.8 Pengujian Konsistensi Data	70
4.9 Pengujian Fungsionalitas	70
4. 10 Komunikasi Antar Aplikasi Dalam Sistem.....	71
BAB V KESIMPULAN DAN SARAN	73
5.1 Kesimpulan	73
5.2 Saran.....	74
DAFTAR PUSTAKA	75

DAFTAR GAMBAR

Gambar 2. 1 <i>Python TCP Server</i>	11
Gambar 2. 2 <i>Python TCP Client</i>	11
Gambar 2. 3 <i>Python UDP Server</i>	12
Gambar 2. 4 <i>Python UDP Client</i>	12
Gambar 2.5 <i>Arsitektur Jaringan World Wide Web</i>	15
Gambar 2.6 <i>Arsitektur Jaringan Email</i>	15
Gambar 2.7 <i>Arsitektur Jaringan XMPP</i>	16
Gambar 2. 8 <i>Stanza Message XMPP</i>	19
Gambar 2. 9 <i>Stanza Presence XMPP</i>	20
Gambar 2. 10 <i>Stanza IQ Get Roster XMPP</i>	21
Gambar 2. 11 <i>Stanza IQ Result Get Roster XMPP</i>	21
Gambar 2. 12 <i>Stanza IQ Set Roster XMPP</i>	21
Gambar 2. 13 <i>Stanza IQ Result Set Roster XMPP</i>	22
Gambar 3.1 <i>Flowchart Tahapan Penelitian Pembuatan Middleware Komunikasi Menggunakan XMPP dan Arsitektur microservice</i>	26
Gambar 3.2 <i>Arsitektur Microservice Middleware Komunikasi dengan XMPP</i> ...	27
Gambar 3.3 <i>Activity Diagram Komponen Tracker</i>	28
Gambar 3.4 <i>Activity Diagram Komponen Manager</i>	29
Gambar 3.5 <i>Activity Diagram Komponen Relay</i>	30
Gambar 3.6 <i>Activity Diagram Komponen User</i>	31
Gambar 3.7 <i>Relasi Antara Tracker dengan Komponen Lainnya</i>	32
Gambar 3.8 <i>Relasi Antara Relay dan Manager</i>	32
Gambar 3.9 <i>Relasi Antara User dengan Manager</i>	32
Gambar 3.10 <i>Relasi Antara Relay dan User</i>	33
Gambar 3.11 <i>Relasi Antar Relay</i>	33
Gambar 3. 12 <i>Stanza Message XMPP</i>	33
Gambar 3. 13 <i>Stanza Message JSON</i>	34
Gambar 3.14 <i>Database Pada Komponen Tracker</i>	34
Gambar 3.15 <i>Database Pada Komponen Manager</i>	35
Gambar 3.16 <i>Database Pada Komponen Relay</i>	36

Gambar 3.17 <i>Database</i> Pada Komponen <i>User</i>	36
Gambar 4. 1 Implementasi Relasi Antar Komponen	39
Gambar 4. 2 Kelas untuk Menjadikan Komponen Sebagai <i>Server</i>	40
Gambar 4. 3 Utilitas untuk Mengelola Waktu Menjadi <i>Timestamp</i>	41
Gambar 4. 4 Diagram Lengkap Komponen <i>Tracker</i>	41
Gambar 4. 5 <i>Tracker Import Starter</i>	42
Gambar 4. 6 <i>Tracker Accept Connection</i>	42
Gambar 4. 7 Hasil <i>Tracker Accept Connection</i>	43
Gambar 4. 8 <i>Tracker</i> Fungsi Menangani Komponen yang Terkoneksi.....	43
Gambar 4. 9 Diagram Komponen <i>Manager</i>	44
Gambar 4. 10 <i>Manager Import Starter</i>	45
Gambar 4. 11 <i>Manager</i> Koneksi ke <i>Tracker</i> dan Menjadi <i>Server</i>	45
Gambar 4. 12 Hasil Koneksi ke <i>Tracker</i> dan Menjadi <i>Server</i>	45
Gambar 4. 13 <i>Manager Accept Connection</i>	46
Gambar 4. 14 Hasil <i>Manager Accept Connection</i>	46
Gambar 4. 15 <i>Manager</i> Mengelola Autentikasi.....	47
Gambar 4. 16 <i>Manager</i> Logika Pengelolaan Registrasi <i>User</i>	47
Gambar 4. 17 Hasil <i>Manager</i> Mengelola Registrasi <i>Username</i> Terpakai	48
Gambar 4. 18 Hasil <i>Manager</i> Mengelola Data <i>User</i> Kosong atau Ada Spasi.....	48
Gambar 4. 19 Hasil <i>Manager</i> Mengelola <i>User</i> Berhasil Registrasi	48
Gambar 4. 20 <i>Manager</i> Logika Pengelolaan <i>Login User</i>	49
Gambar 4. 21 Hasil <i>Manager</i> Mengelola <i>User Login Password</i> Salah.....	49
Gambar 4. 22 Hasil <i>Manager</i> Mengelola <i>User Login Username</i> Salah	49
Gambar 4. 23 Hasil <i>Manager</i> Mengelola <i>User Login</i> Berhasil	50
Gambar 4. 24 <i>Manager</i> Logika Pengelolaan <i>NCIR</i>	50
Gambar 4. 25 Hasil <i>Manager</i> Mengelola <i>NCIR</i>	50
Gambar 4. 26 <i>Manager</i> Logika Pengelolaan <i>ECIR</i>	50
Gambar 4. 27 Hasil <i>Manager</i> Mengelola <i>ECIR</i>	50
Gambar 4. 28 <i>Manager</i> Mengelola <i>Roster</i>	51
Gambar 4. 29 <i>Manager</i> Logika Pengelolaan <i>Get Roster</i>	51

Gambar 4. 30 <i>Manager</i> Logika <i>Get Roster</i> Berdasarkan <i>Subscription</i> yang Sesuai	51
Gambar 4. 31 Hasil <i>Manager</i> Mengelola <i>Get Roster</i>	51
Gambar 4. 32 <i>Manager</i> Logika <i>Set</i> atau <i>Update Roster</i>	52
Gambar 4. 33 <i>Manager</i> Logika Penentu Antara <i>Set</i> atau <i>Update Roster</i>	52
Gambar 4. 34 Hasil <i>Manager</i> Pengelolaan <i>Set Roster</i>	53
Gambar 4. 35 Hasil <i>Manager</i> Mengelola <i>Update Roster</i>	53
Gambar 4. 36 <i>Manager</i> Logika Menghapus <i>Roster</i>	53
Gambar 4. 37 Hasil <i>Manager</i> Mengelola Hapus <i>Roster</i>	53
Gambar 4. 38 <i>Manager</i> Pengelolaan <i>Presence</i>	54
Gambar 4. 39 <i>Manager</i> Logika Pengelolaan <i>Init Presence</i>	54
Gambar 4. 40 Hasil <i>Manager</i> Mengelola <i>Init Presence</i>	54
Gambar 4. 41 <i>Manager</i> Pengelolaan <i>Update Bio User</i>	54
Gambar 4. 42 <i>Manager</i> Logika <i>Update Bio User</i>	55
Gambar 4. 43 Hasil <i>Manager</i> Pengelolaan <i>Update Bio User</i>	55
Gambar 4. 44 <i>Manager</i> Logika <i>Directed Presence</i>	55
Gambar 4. 45 Hasil <i>Manager</i> Mengelola <i>Directed Presence</i>	55
Gambar 4. 46 <i>Manager</i> Pengelolaan <i>Logout User</i>	56
Gambar 4. 47 <i>Manager</i> Logika Pengelolaan <i>Logout User</i>	56
Gambar 4. 48 Hasil <i>Manager</i> Mengelola <i>Logout User</i>	56
Gambar 4. 49 Diagram Komponen <i>Relay</i>	57
Gambar 4. 50 <i>Relay</i> Konek ke <i>Tracker</i>	57
Gambar 4. 51 Hasil <i>Relay</i> Konek ke <i>Tracker</i>	57
Gambar 4. 52 <i>Relay</i> Logika Konek ke <i>Manager</i> , Konfigurasi dengan <i>Relay</i> Lainnya, dan Menjadi <i>Server</i>	58
Gambar 4. 53 Hasil <i>Relay</i> Konek ke <i>Manager</i>	58
Gambar 4. 54 Hasil <i>Relay</i> Konek ke <i>Relay</i> Lainnya.....	58
Gambar 4. 55 Hasil <i>Relay</i> Menyimpan Data <i>User</i> yang Terkoneksi Pada <i>Relay</i> Lainnya.....	59
Gambar 4. 56 <i>Relay</i> <i>Accept Connection</i>	59
Gambar 4. 57 Hasil <i>Relay</i> <i>Accept Connection</i> <i>Relay</i> Lainnya.....	59
Gambar 4. 58 Hasil <i>Relay</i> <i>Accept Connection</i> <i>User</i>	60

Gambar 4. 59 <i>Relay</i> Logika Pengelolaan <i>User</i> Baru dari <i>Relay</i> Lainnya.....	60
Gambar 4. 60 Hasil <i>Relay</i> Mengelola <i>User</i> Baru dari <i>Relay</i> Lainnya	60
Gambar 4. 61 <i>Relay</i> Logika Pengelolaan <i>User</i> Terputus dari <i>Relay</i> Lainnya	60
Gambar 4. 62 Hasil <i>Relay</i> Mengelola <i>User</i> Terputus dari <i>Relay</i> Lainnya.....	61
Gambar 4. 63 <i>Relay</i> Logika Mengirimkan <i>User</i> Terputus	61
Gambar 4. 64 <i>Relay</i> Logika Pengelolaan <i>Stanza Message</i> dari <i>User</i>	62
Gambar 4. 65 Hasil <i>Relay</i> Mengelola <i>Stanza Message</i> <i>User</i> Tujuan Tidak Terdaftar.....	62
Gambar 4. 66 Hasil <i>Relay</i> Mengelola <i>Stanza Message</i> <i>User</i> Berada Pada <i>Relay</i> yang Sama	62
Gambar 4. 67 Hasil <i>Relay</i> Mengelola <i>Stanza Message</i> Untuk <i>Relay</i> Lainnya	63
Gambar 4. 68 <i>Relay</i> Logika Pengelolaan <i>Stanza Message</i> Dari <i>Relay</i> Lainnya...	63
Gambar 4. 69 Hasil <i>Relay</i> Meneruskan <i>Stanza Message</i> Kepada <i>User</i> Tujuan....	63
Gambar 4. 70 Diagram Komponen <i>User</i>	64
Gambar 4. 71 Standar Pesan Menambahkan Roster	66
Gambar 4. 72 Standar Pesan Meminta Daftar Roster	66
Gambar 4. 73 Standar Pesan Memperbarui Alias Roster.....	67
Gambar 4. 74 Standar Pesan Menghapus Roster	68
Gambar 4. 75 Standar Pesan Inisialisasi <i>Presence</i>	68
Gambar 4. 76 Standar Pesan <i>Directed Presence</i>	68
Gambar 4. 77 Standar Pesan Memperbarui Bio.....	69
Gambar 4. 78 Standar Pesan Melakukan <i>Logout</i>	69
Gambar 4. 79 Standar Pesan Mengirimkan Pesan Kepada <i>User</i> Lain.....	69

DAFTAR TABEL

Table 2.1 Perbedaan Arsitektur Jaringan	16
Table 2.2 Perbandingan JSON dan XML GET Without GZIP	23
Table 2.3 Perbandingan JSON dan XML GET With GZIP	23
Table 2.4 Perbandingan JSON dan XML POST	24
Table 2.5 Perbandingan JSON dan XML PUT	24
Table 2.6 Perbandingan JSON dan XML DELETE	25

BAB I PENDAHULUAN

1.1 Latar Belakang

Komunikasi di zaman sekarang dapat dilakukan di mana saja dan kapan saja, tidak memedulikan jarak dan waktu pengiriman selama memiliki jaringan internet. Komunikasi dilakukan melalui gawai atau komputer setiap orang, namun sayangnya komunikasi tidak dapat terjadi jika perangkat memiliki jaringan yang berbeda. Cara menanganinya adalah dengan aplikasi.

Aplikasi membutuhkan waktu untuk pembuatannya, mulai dari membuat tampilan depan dan juga alur pengiriman data yang dilakukan oleh aplikasi tersebut. Aplikasi juga memiliki banyak fitur atau kebutuhan sesuai dengan tujuannya masing-masing, sehingga untuk mengembangkan fitur komunikasi dalam aplikasi pasti membutuhkan waktu tersendiri untuk memikirkan alur pengiriman data maupun tampilannya. Saat mengembangkan aplikasi, pengembang juga memiliki pilihan untuk menggunakan aplikasi penghubung atau *middleware*.

Middleware adalah aplikasi penghubung yang dikembangkan untuk tujuan tertentu, contohnya komunikasi. *Middleware* membantu pengembang dalam mempercepat pengembangan aplikasinya karena pengembang hanya perlu memikirkan tampilan, mengirimkan data ke *middleware*, dan menerima data dari *middleware*. Hal ini mengartikan bahwa *middleware* akan mengurus permasalahan alur pengiriman data agar sesuai dengan tujuannya.

Alur pengiriman data untuk tujuan komunikasi secara *real-time* memiliki sebuah protokol yang disebut XMPP. XMPP adalah teknologi *open source* untuk komunikasi secara *real-time* dan memiliki berbagai kegunaan. XMPP tercatat dalam dokumentasi *Internet Engineering Task Force* (IETF) pada RFC 6120 untuk teknologi inti dan RFC 6121 untuk *Instant Messaging and Presence*.

RFC 6120 membahas mengenai teknologi inti seperti arsitektur dari XMPP. XMPP memiliki arsitektur *monolith*, format data XML, dan tipe koneksi yang digunakan adalah *long-lived TCP connections* (P. Saint-Andre, 2011). Arsitektur *monolith* adalah sistem yang dikembangkan menjadi satu aplikasi utuh, jika aplikasi dideploy pada *server* dan *server* mati maka layanan juga akan terhenti seluruhnya. Cara mengatasinya adalah menggunakan arsitektur *microservice* yaitu layanan dipisahkan menjadi bagian kecil sesuai dengan bagiannya. (Chandler Harris, 2024). Format data XML adalah format data yang tidak efisien jika dibandingkan dengan format data JSON (Breje et al., 2018). Koneksi tcp akan memakan sumber daya *server*, maka jika tipenya *long-lived TCP connections* dan banyak yang melakukan koneksi ke *server* akan mengakibatkan *server* menjadi lambat atau berhenti (Maxwin, 2021).

Penggunaan XMPP sebagai jalur penghubung komunikasi dalam aplikasi sudah sering diterapkan. Contohnya pembuatan aplikasi *web instant messaging* (Gupta et al., 2021).

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan, dapat dirumuskan beberapa permasalahan yaitu:

1. Bagaimana cara membuat *middleware* komunikasi menggunakan protokol XMPP dan *socket programming*?
2. Bagaimana cara mengubah arsitektur *monolith* menjadi *microservice*?

1.3 Batasan Masalah

Penelitian ini akan memiliki beberapa batasan masalah diantaranya adalah:

1. Penelitian hanya akan berfokus kepada pembuatan *middleware* komunikasi.
2. Penelitian tidak akan membahas topik keamanan data.
3. Pengujian pada penelitian akan berfokus pada pengujian fungsionalitas dari *middleware* komunikasi.

1.4 Tujuan Penelitian

Penelitian memiliki tujuan yaitu membuat *middleware* komunikasi menggunakan XMPP dan arsitektur *microservice*.

1.5 Manfaat Penelitian

Penelitian ini dapat memberikan beberapa manfaat yang mana diantaranya adalah:

1. Mengetahui cara pembuatan *middleware* komunikasi menggunakan XMPP dan arsitektur *microservice*, yang fiturnya dapat diperluas.
2. Mengetahui interaksi antar *service* pada bentuk arsitektur *microservice* dalam *middleware*.
3. *Middleware* dapat dikembangkan sesuai dengan *best practice* setiap komponen atau kasus yang dihadapi.

BAB II KAJIAN PUSTAKA

2.1 Jaringan Komputer

Menurut Melwin Syafrizal jaringan komputer adalah hubungan interkoneksi yang menggunakan media kabel atau tanpa kabel antara dua komputer atau lebih yang saling terhubung. Jaringan komputer terbentuk karena perangkat komputasi saling terhubung satu sama lain melalui suatu koneksi tertentu. Jaringan komputer ini berfungsi untuk jalur pertukaran data yang akan dilakukan oleh sekumpulan perangkat komputasi yang saling terhubung tersebut. Hubungan koneksi antar perangkat ini dapat terkoneksi secara fisik melalui kabel atau lainnya, maupun terhubung secara nirkabel seperti gelombang radio atau internet.

Peran jaringan komputer sangat besar hal ini dikarenakan tanpa adanya jaringan komputer maka perangkat komputasi tidak dapat saling bertukar data yang dimiliki atau yang diinginkan. Contoh pertukaran informasi seperti ketika seseorang ingin mencari sebuah bahan bacaan maka ia dapat mencarinya melalui internet menggunakan mesin pencari seperti Google Chrome ataupun seseorang ingin mencari temannya melalui media sosial seperti Instagram atau Facebook. Arsitektur yang digunakan biasanya memiliki model *client-server*, dimana setiap keinginan *client* akan dipenuhi oleh *server* selama permintaan tersebut dapat ditangani oleh *server* (Amira, 2021).

2.2 Socket

Socket adalah titik komunikasi dari lalu lintas komunikasi antar proses di dalam sebuah jaringan komputer. Hampir semua komunikasi antar komputer sekarang berdasarkan protokol internet, oleh karena itu hampir semua *socket* di jaringan komputer adalah *socket internet*. Sebuah *socket* memiliki alamat dan alamat tersebut terdiri dari kombinasi alamat IP dan juga nomor *port*. Setiap paket yang dikirimkan atau diterima pasti akan melalui *socket* yang terdiri dari dua kombinasi tersebut. *Socket* memiliki beberapa protokol diantaranya adalah protokol TCP dan protokol UDP (Shafiei et al., 2012).

2.2.1 Socket Protokol TCP

Protokol TCP (*Transmission Control Protocol*) adalah protokol yang berorientasi koneksi (*connection-oriented*) sehingga jika ingin mengirimkan data melalui *socket* dengan protokol TCP perangkat yang akan terlibat didalamnya harus saling memiliki koneksi. Sebuah hal yang wajar dikatakan bahwa sebelum mengirimkan data melalui protokol TCP ini wajib hukumnya koneksi antar perangkat dibangun terlebih dahulu. Pada protokol TCP ini pengiriman data bisa dipastikan memiliki kemungkinan risiko data menjadi rusak atau hilang adalah kecil (Shafiei et al., 2012).

Koneksi TCP dapat bertipe *long-lived connections*. TCP dengan tipe koneksi seperti ini tidak akan pernah terputus kecuali jika terjadi error atau sengaja dimatikan. Koneksi yang selalu terhubung akan menyebabkan pengurangan sumber daya sistem dan jika yang melakukan koneksi semakin banyak maka akan mempengaruhi performa dari sistem. Sistem akan menjadi lambat atau mati (Maxwin, 2021).

2.2.2 Socket Protokol UDP

Protokol UDP (*User Datagram Protocol*) adalah sebuah protokol sederhana yang mana pengiriman data melalui protokol UDP ini tidak mewajibkan untuk membangun koneksi terlebih dahulu (*connection-less*) (Shafiei et al., 2012).

Protokol UDP hanya memerlukan alamat IP dan nomor *port* dari perangkat yang dituju. Protokol ini memiliki kecepatan pengiriman data yang lebih cepat dibanding pengiriman data melalui protokol TCP. Namun protokol UDP memiliki kelemahan yaitu protokol ini memiliki sifat “*fire and forget*” yang mana data akan dikirimkan tanpa memperdulikan bahwa data tersebut telah tersampaikan dengan sempurna kepada penerima atau terdapat kecacatan dalam proses pengiriman data (Ben Gorman, 2023).

2.3 IP Address

IP (*Internet Protocol*) *address* atau alamat IP adalah alamat yang memiliki peran sebagai perwakilan dari sebuah koneksi individual baik itu dalam lingkup luas di internet atau dalam lingkup kecil atau lokal (Roberts & Challinor, 2000). Alamat IP pada saat ini memiliki dua tipe yaitu IPv4 dan IPv6. Adapun jenis dari alamat IP adalah IP publik dan IP *private* (Faradilla A., 2023). Bagaimanakah cara komunikasi antara kedua jenis alamat IP ini.

2.3.1 IPv4

IPv4 adalah tipe alamat IP yang umum digunakan dengan panjang 32-bit dan terdiri dari empat oktet yang terpisah oleh titik dua. Setiap oktet memiliki cakupan panjang mulai dari 0 sampai 255. IPv4 memiliki kemungkinan untuk mewakili sekitar 4,3 miliar perangkat berbeda diseluruh dunia. Contoh dari IPv4 adalah 123.142.132.200 (Faradilla A., 2023).

2.3.2 IPv6

IPv6 adalah tipe alamat IP terbaru yang mana alamat IP ini muncul untuk menggantikan IPv4 dikarenakan alamat IPv4 yang awalnya diharapkan dapat mewakili seluruh perangkat yang ada didunia ini tiba-tiba menjadi sebuah cakupan yang sangat kecil. Hal ini dikarenakan meledaknya teknologi gawai. Permasalahan tersebutlah yang memunculkan sebuah inovasi baru yaitu IPv6.

Tidak seperti IPv4 yang memiliki panjang 32-bit, IPv6 memiliki panjang 128-bit. IPv6 juga dituliskan dalam rangkaian angka heksadesimal dan huruf serta dipisahkan pula oleh titik dua sama seperti IPV4. Contoh dari IPv6 2001:db8:3333:4444:CCCC:DDDD:EEEE:FFFF (Faradilla A., 2023).

2.3.3 Alamat IP Publik

Alamat IP publik adalah sebuah alamat yang mewakili perangkat dalam lingkup lingkungan yang luas yaitu internet. Ketika berkomunikasi melalui internet perangkat wajib hukumnya memakai alamat IP publik karena alamat IP publik adalah alamat unik yang hanya dimiliki oleh satu perangkat diseluruh dunia. Alamat IP publik juga dapat dijadikan sebagai sebuah alamat tampungan

seperti yang digunakan oleh ISP (*Internet Service Provider*) agar satu alamat IP publik dapat dipakai oleh beberapa perangkat sekaligus.

Pemakaian satu alamat IP publik oleh beberapa perangkat sekaligus membuat penghematan ketika krisis alamat IP publik pada IPv4. Ketika satu alamat IP publik dipakai secara bersama-sama oleh beberapa perangkat, cara alamat IP publik menentukan perangkat mana yang memberikannya permintaan itulah yang harus menerima balasan atas permintaan tersebut adalah dengan cara pemberian sebuah alamat khusus yaitu alamat IP *private*. Contoh dari alamat IP publik 103.8.90.201 (Biznet News, 2023b).

2.3.4 Alamat IP *Private*

Alamat IP *private* adalah sebuah alamat yang mewakili perangkat dalam lingkup lingkungan yang kecil atau lokal yaitu sebuah alamat yang hanya dapat diakses dalam satu area yang terhubung oleh sebuah alamat IP publik dan tidak dapat langsung diakses melalui internet. Alamat IP *private* membantu IP publik dalam membedakan setiap perangkat yang terhubung ke IP publik. Adapun tipe untuk IP *private* adalah sebagai berikut:

1. 10.0.0.0 hingga 10.255.255.255.
2. 172.16.0.0 hingga 172.16.255.255.
3. 192.168.0.0 hingga 192.168.255.255 (Faiz, 2023).

2.3.5 Komunikasi Antar Alamat IP

Saat melakukan proses pertukaran data pastinya perlu komunikasi antar alamat IP sehingga data dapat dikirimkan oleh pengirim dan diterima oleh penerima. Cara berkomunikasi antar alamat IP publik dengan publik, alamat IP publik dengan *private*, alamat IP *private* dengan publik, alamat *private* dengan *private*.

1. Publik dengan publik

Komunikasi antara IP publik dengan publik sungguh sangat dapat dilakukan. Hal ini dikarenakan alamat IP publik mewakili perangkat dalam lingkup luas sehingga sudah dapat dipastikan mana perangkat yang mengirim dan mana perangkat yang menerima.

2. Publik dengan *private*

Komunikasi antara IP publik dengan *private* dapat dilakukan ketika IP *private* diwakili oleh sebuah IP publik dalam jaringannya yang mana ketika data dikirimkan ke IP publik perwakilannya maka data tersebut dapat diteruskan kepada IP *private*. Hal ini disebut dengan NAT (*Network Address Translator*).

3. *Private* dengan publik

Komunikasi antara IP *private* dengan publik dapat dilakukan ketika IP *private* ingin melakukan pengiriman data, maka IP *private* wajib memberikan data kepada IP publik yang mewakilkannya didalam lingkup internet luas.

4. *Private* dengan *private*

Komunikasi langsung antara IP *private* dengan *private* dapat dilakukan jika kedua perangkat tersebut berada dalam jaringan IP publik yang sama. Akan tetapi jika kedua perangkat tersebut berada dalam jaringan IP publik yang berbeda maka harus menggunakan beberapa trik. IP *private* pengirim harus diubah menjadi IP publik dan mengirimkannya kepada IP publik penerima yang mewakili IP *private* perangkat penerima dan seterusnya (Rekhter et al., 1996). Adapun trik tersebut mengalami banyak kendala yang memerlukan banyak konfigurasi dimana belum tentu seseorang memiliki hak penuh untuk melakukan konfigurasi tersebut. Sehingga dapat dikatakan bahwa komunikasi dari alamat IP *private* dengan *private* agak sulit cenderung mustahil jika tidak memiliki kuasa penuh untuk konfigurasi. Salah satu cara agar alamat IP *private* dapat berkomunikasi dengan *private* lainnya adalah dengan menggunakan *relay* (Oikarinen & Reed, 1993).

2.4 Relay

Relay adalah sistem telekonferensi, yang sangat cocok untuk dijalankan di banyak mesin secara terdistribusi. Pengaturan yang dipakai adalah sebuah *server* menjadi titik keterhubungan *client*, melakukan pengiriman pesan atau fungsi yang lainnya (Oikarinen & Reed, 1993). *Relay* dibuat khusus untuk melewati semua konfigurasi yang harus dilakukan ketika alamat IP *private* ingin berkomunikasi

dengan *private* lainnya. Jalur ini dibuat dengan cara menjalankan sebuah sistem pada alamat IP publik yang dapat diakses oleh semua. Kemudian alamat IP *private* dapat terhubung *relay* dengan diwakili oleh alamat IP publik yang mewakili dirinya.

Bisa diperhatikan bahwa *relay* tidaklah mencoba untuk membuat koneksi terhadap alamat IP *private* melainkan IP *private* lah yang membuat koneksi terhadap *relay*. Hal ini dapat berhasil karena *relay* adalah sebuah alamat IP publik yang dapat dikoneksikan oleh siapa saja tanpa perlu melakukan konfigurasi rumit seperti pada router atau sejenisnya.

Saat koneksi sudah terjadi dua kali antara *relay* dengan IP *private* maka kedua IP *private* dapat dikatakan terhubung melalui jalur *relay*. Sehingga kedua IP *private* sudah dapat melakukan proses pertukaran data sesuka hati mereka selama jalur *relay* masih tersedia. Jadi dapat dikatakan bahwa *relay* berfungsi sebagai jalur jalannya data antara IP *private* (Oikarinen & Reed, 1993).

2.5 *Middleware*

Middleware adalah komponen perangkat lunak yang menjembatani aplikasi, data, dan pengguna. Membantu menyederhanakan pengembangan aplikasi dan membuat pengembang bekerja secara lebih efisien karena mereka dapat berfokus pada pengembangan fitur perangkat lunak yang diinginkan (Revou, 2024).

2.6 *Monolithic*

Arsitektur *monolith* atau monolitik adalah model tradisional dari program perangkat lunak. Arsitektur ini dibangun sebagai satu kesatuan yang mandiri dan independen dari aplikasi lain. Kelebihan dari arsitektur ini adalah penerapan yang mudah, pengembangan hanya memerlukan satu basis kode, kemudahan dalam *debugging*. Kelemahan dari arsitektur ini adalah aplikasi monolitik yang besar akan sulit untuk pengembangannya dan lambat, tidak bisa menskalakan komponen individu atau layanan, jika terdapat masalah pada suatu modul maka seluruh aplikasi atau layanan akan terpengaruh, ketika ingin mengubah basis bahasa pemrograman harus menulis ulang aplikasi dari awal, ketika ada

perubahan pada suatu fitur maka harus dimulai ulang seluruh aplikasi dari awal (Chandler Harris, 2024).

2.7 *Microservice*

Arsitektur *microservice* adalah model arsitektur yang mengandalkan serangkaian layanan yang dapat diterapkan secara independen. Setiap layanan memiliki logika bisnis dan database tersendiri untuk mencapai tujuan layanannya. Kelebihan arsitektur ini adalah kemudahan dalam menskalakan aplikasi, kemudahan dalam maintenance aplikasi dan melakukan pengetestan, penerapan yang mandiri, fleksibilitas teknologi yang akan digunakan untuk layanan tertentu, ketika memulai ulang suatu layanan tidak akan mempengaruhi layanan yang lainnya. Kelemahan arsitektur ini adalah jika tidak ada yang mengepalai pengembangan maka pengembangan aplikasi akan menjadi lambat, harga yang harus dibayarkan untuk infrastruktur semakin besar, *debugging* menjadi lebih kompleks (Chandler Harris, 2024).

2.8 VPS (*Virtual Private Server*)

VPS adalah mesin yang mengurus semua perangkat lunak dan data yang diperlukan untuk menjalankan sistem. *Server* ini disebut virtual karena hanya mengonsumsi sebagian dari sumber daya fisik yang mendasari *server* yang dikelola oleh penyedia pihak ketiga. Namun, pengguna mendapatkan akses ke sumber daya khusus pada perangkat keras tersebut. Adapun keunggulan atau kelebihan dari VPS adalah sebagai berikut (Biznet News, 2023a):

1. Mampu menangani trafik yang tinggi.
2. Sumber daya komputasi tidak terbagi-bagi.
3. Meningkatkan kinerja dari sistem.
4. Cukup memikirkan pembuatan aplikasi tanpa kesulitan untuk konfigurasi *server*.
5. Dapat diakses melalui jarak jauh atau *remote*.
6. Harga relatif terjangkau atau murah.
7. Penyedia layanan VPS sudah sangat banyak baik di Indonesia maupun luar negeri, sehingga memiliki banyak pilihan untuk menentukan VPS terbaik sesuai kebutuhan yang dimiliki (Biznet News, 2023a).

2.9 Python 3

Python 3 adalah salah satu bahasa pemrograman *open source* yang dapat digunakan untuk membantu dalam penangan program dengan menggunakan *socket*. Python 3 memiliki kemampuan untuk mengimplementasikan *socket* bertipe TCP dan juga UDP. Penerapan tipe TCP tentu berbeda dengan UDP. TCP pada *socket* Python 3 diwakili dengan `SOCK_STREAM` sedangkan UDP pada *socket* Python 3 diwakili dengan `SOCK_DGRAM`. Contoh dari pembuatan *socket* tipe TCP dan UDP pada Python 3 dengan model *client-server* adalah sebagai berikut (Python, 2024).

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("0.0.0.0", 5000))
server.listen()

client_socket, client_address = server.accept()

data = client_socket.recv(1024).decode("utf-8")
print(data)

client_socket.send(("Connection success!".encode("utf-8")))
```

Gambar 2. 1 Python TCP Server

```
import socket

SERVER_ADDRESS = "0.0.0.0"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_ADDRESS, PORT))

client.send("Hello Server!".encode("utf-8"))

data = client.recv(1024).decode("utf-8")
print(data)
```

Gambar 2. 2 Python TCP Client

Kode TCP diatas adalah cara arsitektur *client-server* TCP melakukan pengiriman data (Verma, 2023).

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(("0.0.0.0", 5000))

data, address_initiate = server.recvfrom(1024)
data = data.decode("utf-8")
print(data)

server.sendto("Hello Server!".encode("utf-8"), address_initiate)
```

Gambar 2. 3 Python UDP Server

```
import socket

SERVER_ADDRESS = "0.0.0.0"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

SERVER_TARGET = (SERVER_ADDRESS, PORT)
client.sendto("Hello Server!".encode("utf-8"), SERVER_TARGET)

data, address_initiate = client.recvfrom(1024)
data = data.decode("utf-8")
print(data)
```

Gambar 2. 4 Python UDP Client

Kode UDP diatas adalah cara arsitektur *client-server* UDP melakukan pengiriman data (Virga, 2020).

2.10 XMPP

Extensible Messaging Presence And Protocol (XMPP) adalah teknologi terbuka untuk komunikasi secara *real-time*, menggunakan *Extensible Markup Language* (XML) sebagai format dasar untuk pertukaran informasi. XMPP menyediakan cara untuk melakukan pengiriman potongan-potongan kecil XML dari satu entitas ke entitas lain dalam waktu yang hampir mendekati instan.

XMPP digunakan untuk jangkauan variasi aplikasi yang beragam. Pembahasan lebih dalam mengenai variasi penggunaan XMPP dapat dibagi menjadi dua hal yaitu dari sisi *services* dan dari sisi aplikasi. *Services* dalam XMPP terbagi menjadi dua bagian yaitu yang utama dan *extension*. Pembahasan

mengenai *services* yang utama dipublikasikan oleh *Internet Engineering Task Force* (IETF) pada situs <http://ietf.org> disebut sebagai seri RFC (*Request For Comments*), dan mengenai *service* berupa *extension* dipublikasikan oleh *XMPP Standards Foundation* (XSF) pada situs <http://xmpp.org> disebut sebagai seri XEP (*XMPP Extensible Protocol*). Aplikasi adalah program perangkat lunak yang disebarkan melalui skenario ketertarikan umum secara individu ataupun organisasi (Saint-Andre et al., 2009).

2.10.1 Services

Service adalah fitur atau fungsi yang dapat digunakan oleh berbagai aplikasi apapun. Implementasi XMPP biasanya menyediakan layanan inti berupa:

1. *Channel Encryption*

Didefinisikan pada [RFC 3920] yang mana adalah kemampuan enkripsi koneksi untuk klien dan *server*, atau antara dua *server* yang berhubungan.

2. *Authentication*

Didefinisikan pada [RFC 3920] yang mana adalah kemampuan untuk mendeteksi atau memastikan entitas yang mencoba berkomunikasi melalui jaringan sudah diautentikasi oleh *server* (bertindak seperti penjaga pada akses jaringan).

3. *Presence*

Didefinisikan pada [RFC 3921] yang mana adalah kemampuan untuk mencari tahu tentang kondisi entitas pada suatu jaringan. Fitur *presence* didasarkan pada ikatan antara entitas yang sudah saling terkoneksi untuk menjaga privasi pengguna.

4. *Contact Lists*

Didefinisikan pada [RFC 3921] yang mana adalah kemampuan untuk menyimpan daftar kontak atau *roster* pada XMPP *server*.

5. *One-to-one Messaging*

Didefinisikan pada [RFC 3920] yang mana adalah sebuah kemampuan pada entitas untuk mengirimkan pesan kepada entitas lain (Saint-Andre et al., 2009).

2.10.2 Aplikasi

Banyak macam aplikasi yang dapat dibuat menggunakan XMPP sebagai dasar pembuatannya. Contoh dari aplikasi yang dapat dibuat adalah:

1. *Instant Messaging.*

Aplikasi *instant messaging* klasik yang familiar dibanyak orang adalah kombinasi dari tiga layanan yaitu, *presence*, *contact list*, dan *one-to-one messaging*.

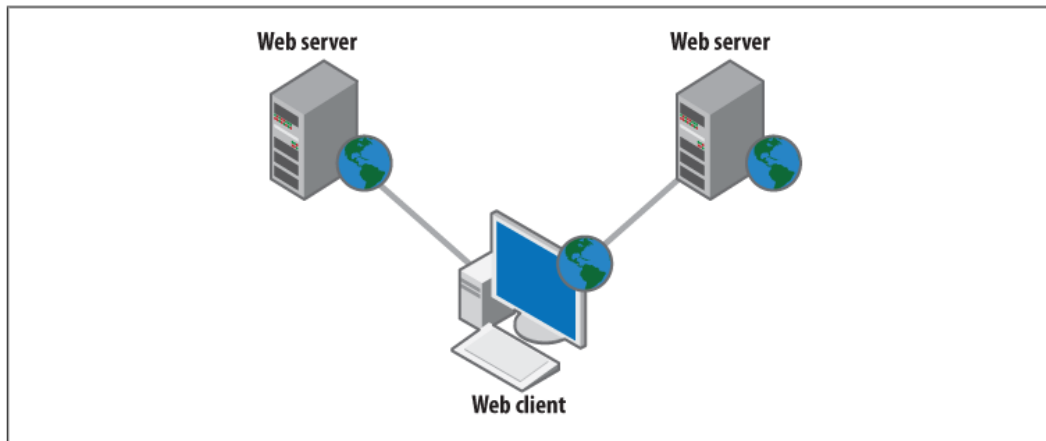
2. *Middleware and Cloud Computing*

Banyak perusahaan dan kelompok peneliti secara aktif bekerja dengan sistem berbasis XMPP untuk layanan komputasi, *lightweight middleware*, dan manajemen infrastruktur *cloud computing* (Saint-Andre et al., 2009).

2.10.3 Arsitektur XMPP

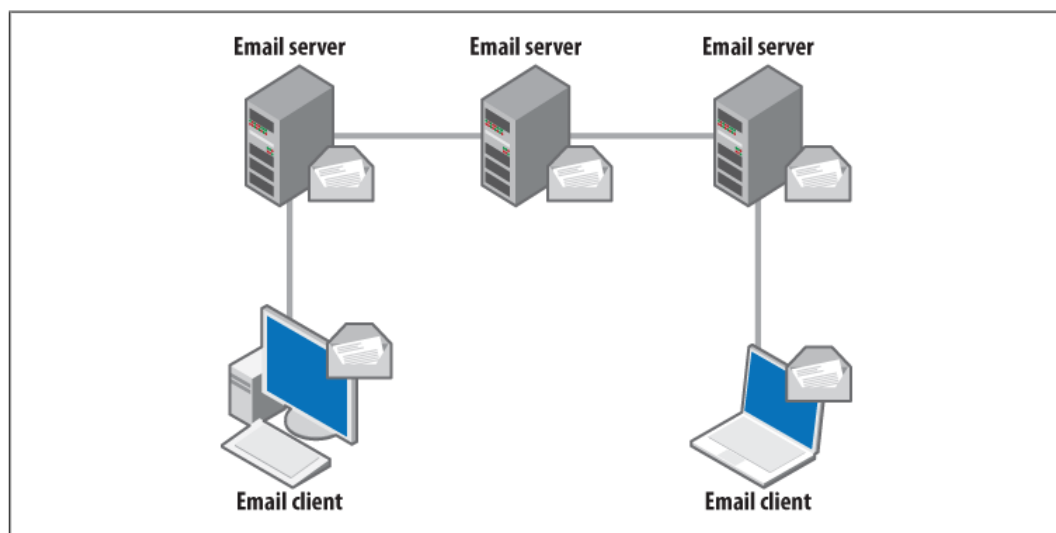
Semua teknologi yang ada dalam internet memiliki cara untuk saling terhubung dan berkomunikasi. Teknologi XMPP menggunakan arsitektur *client-server* yang terdesentralisasi serupa dengan arsitektur yang digunakan dalam *World Wide Web* dan jaringan *email*. Adapun perbedaan penting dalam arsitektur *World Wide Web*, *email*, dan XMPP akan dijelaskan dibawah.

Ketika menggunakan website, mesin pencari akan terkoneksi ke web server, tetapi antara web server biasanya tidak saling terkoneksi untuk menyelesaikan transaksi. Sebaliknya, HTML halaman *website* mungkin merujuk ke web server yang lainnya dan mesin pencari akan membuka session dengan web server tersebut untuk memuat halaman penuhnya. Dengan demikian *website* biasanya tidak memiliki koneksi antar-domain (*inter-domain*).



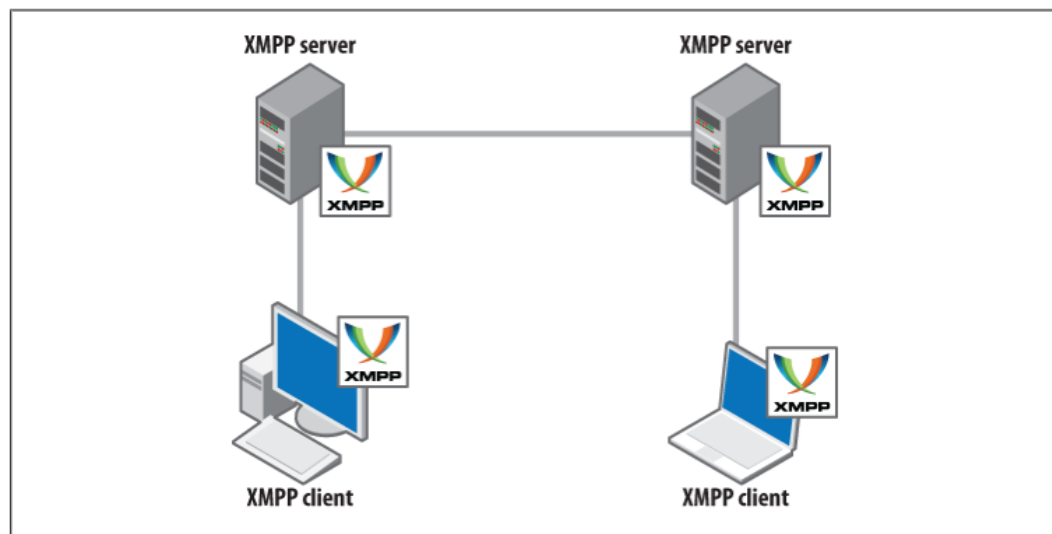
Gambar 2.5 Arsitektur Jaringan *World Wide Web*

Ketika mengirimkan *email* atau pesan ke salah satu kontak yang memiliki domain yang berbeda (xx@gmail.com ke xx@yahoo.com), pertama kali *email client* akan terkoneksi kepada domain utama *email server* pengirim, kemudian akan mencari rute menuju ke domain kontak yang dituju. Dengan demikian, tidak seperti *website*, sistem pada arsitektur *email* terdiri dari jaringan *server* gabungan. Bagaimanapun *email* yang dikirimkan memiliki kemungkinan akan melewati beberapa *email server* perantara sebelum mencapai tujuan akhir yaitu kontak yang ingin dituju. Dengan demikian jaringan *email* menggunakan beberapa lompatan antar *server* untuk mengirimkan pesan.



Gambar 2.6 Arsitektur Jaringan *Email*

Arsitektur XMPP lebih mirip dengan arsitektur *email* karena mampu melakukan koneksi antar domain (*inter-domain connections*). Ketika akan mengirimkan pesan kepada salah satu kontak dalam jaringan XMPP, maka pesan akan terkoneksi dengan domain miliknya kemudian *server* miliknya akan langsung terkoneksi pada *server* kontak yang dituju tanpa harus melalui *server* perantara. Koneksi langsung antar *server* ini memiliki kegunaan penting seperti mencegah *address spoofing* dan bentuk spam tertentu (Saint-Andre et al., 2009).



Gambar 2.7 Arsitektur Jaringan XMPP

Table 2.1 Perbedaan Arsitektur Jaringan

Arsitektur	<i>Interdomain Connections</i>	<i>Multiple Hops</i>
<i>World Wide Web</i>	<i>No</i>	-
<i>Email</i>	<i>Yes</i>	<i>Yes</i>
XMPP	<i>Yes</i>	<i>No</i>

2.10.4 Pengalaman XMPP

Pada jaringan XMPP tentu saja akan mempunyai banyak entitas. Cara membedakan satu entitas dengan entitas lainnya adalah menggunakan alamat unik yang dimiliki tiap entitas. Adapun alamat unik dalam XMPP disebut sebagai

JabberID (JID). JabberID memiliki format yang mirip seperti format alamat pada email yaitu *user@dns.com*. Pengalamatan pada JabberID ada dua tipe yaitu *bare JID* dan *full JID*. Perbedaan diantara keduanya adalah *bare JID* ditulis seperti *user@dns.com* sedangkan *full JID* ditulis seperti *user@dns.com/resource* (Saint-Andre et al., 2009).

2.10.5 DNS (*Domain Name System*)

DNS adalah sebuah bentuk sederhana dari alamat IP. Penggunaan DNS dimaksudkan agar memudahkan mengingat alamat IP dengan menggunakan sebuah kata dibandingkan dengan nomor dari alamat IP itu sendiri. Contoh alamat IP untuk sebuah *website* pemberitahuan berita adalah 102.56.72.109 akan lebih sulit dihafal dibandingkan dengan <https://beritakumilikku.com>. Ketika DNS ini digunakan untuk membuat sebuah JabberID pada pengalamatan XMPP akan menjadi seperti *user@beritakumilikku.com* (Saint-Andre et al., 2009).

2.10.6 *Resource*

Resource yang tertera ketika JabberID adalah *full JabberID* adalah sebuah indikator untuk membedakan perangkat dari pengguna. Contoh pengguna atas nama Fajar dari *website* beritakumilikku.com memiliki dua perangkat yang terkoneksi ke dalam sistem XMPP, maka *full JabberID* untuk masing-masing perangkat adalah *Fajar@beritakumilikku.com/perangkatKerja1* adalah JabberID pada perangkat pertama dan *Fajar@beritakumilikku.com/perangkatKerja2* adalah JabberID pada perangkat kedua. Pada penamaan *resource* atau perangkat dalam sistem XMPP memiliki ketentuan yang berbeda-beda bergantung pada layanan sistem XMPP tersebut. Penamaan *resource* ada yang bersifat otomatis tidak dapat ditentukan oleh pengguna atau membebaskan pengguna untuk menentukan nama *resource* tersebut, selama *full JabberID* tidak menyamai dengan *full JabberID* yang sudah ada (Saint-Andre et al., 2009).

2.10.7 Komunikasi XMPP

Komunikasi dalam XMPP dibentuk dengan koneksi *long-lived TCP connections*, artinya koneksi antara *client-server* dalam XMPP tidak akan pernah terputus selama masih ingin melakukan pertukaran data. Dalam XMPP terdapat sesuatu yang disebut *stanza*. *Stanza* adalah unit standar komunikasi dalam XMPP

mirip seperti *packet* atau pesan didalam protokol jaringan. Beberapa faktor yang menentukan makna *stanza*:

1. Nama dari elemen *stanza* yang mana ada beberapa yaitu *message*, *presence*, *iq (info/query)*. Masing-masing *stanza* ditangani berbeda-beda oleh *server* dan *client*.
2. Nilai dari atribut *type* yang bervariasi ditentukan berdasarkan sedang membicarakan *stanza* yang mana. Hal ini semakin membedakan setiap *stanza* berdasarkan cara pemrosesan oleh penerimanya.
3. *Child element* yang mana mendefinisikan sebuah payload untuk *stanza*. *Payload* mungkin diperlihatkan kepada pengguna atau diproses secara otomatis sebagaimana ditentukan oleh spesifikasi yang mendefinisikan *namespace payload* (Saint-Andre et al., 2009).

2.10.8 Stanza Message

Stanza message adalah metode dasar “*push*” untuk mengirimkan informasi dari satu tempat ke tempat lainnya. *Message* digunakan dalam sistem *instant messaging*, *groupchat*, *alerts and notifications*, dan sistem lainnya. *Message stanza* memiliki lima jenis yang dibedakan berdasarkan atribut *type* yaitu:

1. Normal

Tipe ini mirip seperti pesan *email*. Tipe pesan yang mungkin akan ditanggapi atau tidak setelah dikirim.

2. Chat

Tipe ini adalah bertukar pesan dalam waktu *real-time session* antara dua entitas. Mirip seperti teman yang saling bertukar pesan dalam aplikasi media sosial.

3. Groupchat

Tipe ini adalah bertukar pesan oleh banyak pengguna dalam ruang obrolan, mirip seperti *Internet Relay Chat (IRC)*.

4. Headline

Tipe ini digunakan untuk mengirimkan *alerts and notifications* dan tidak mengharapkan tanggapan apapun dari penerima.

5. Error

Tipe ini adalah untuk mengetahui apakah pesan yang dikirimkan mengalami kendala atau tidak, jika mengalami kendala maka tipe ini akan muncul sebagai tanggapan.

Stanza *message* selain memiliki atribut *type*, juga memiliki atribut *from* dan *to*, dimana atribut *from* menandakan siapakah yang mengirimkan pesan dan atribut *to* menandakan siapakah yang akan menerima pesan yang dikirimkan. Bagian *from* dan *to* akan berisikan JabberID masing-masing pengguna. Perbedaanannya adalah bagian *from* didefinisikan oleh *server* pengirim untuk menghindari *address spoofing*. Stanza *message* juga memuat elemen *payload*. Spesifikasi inti XMPP mendefinisikan *payload* yang sangat dasar, seperti `<body/>` dan `<subject/>`. Contoh sederhana dari *stanza message* sebagai berikut (Saint-Andre et al., 2009).

```
<message
  from="person1@server.org"
  to="friend@server.org"
  type="chat">
  <body>Apa kabar bro?</body>
  <subject>Bertanya</subject>
</message>
```

Gambar 2. 8 Stanza Message XMPP

2.10.9 Stanza Presence

Salah satu ciri khas dari sistem komunikasi *real-time* adalah *presence* (kehadiran). Kegunaan dari *presence* adalah memperlihatkan ketersediaan jaringan dari suatu entitas dan dengan demikian memungkinkan pengguna melihat entitas tersebut sedang dalam jaringan dan dapat melakukan komunikasi. Untuk melihat kehadiran entitas lain pengguna harus melakukan sebuah otorisasi yaitu *presence subscription* dan harus diterima oleh entitas yang ingin dituju. Pengguna yang telah melakukan hal tersebut maka akan secara berkala menerima pembaruan informasi *presence* dari entitas. Prinsip paling dasarnya dari *stanza* ini adalah sebuah indikator untuk mengetahui apakah entitas sedang dalam jaringan atau diluar jaringan (contoh keadaannya adalah “diluar” dan “tidak bisa diganggu”). Pada *stanza* ini juga dapat dilakukan perluasan fitur seperti

memberikan pesan status seperti “sedang berada di kampus” atau “sedang mengemudi”. Contoh dari *stanza presence* adalah:

```
<presence
  from="person1@server.org">
  <show>Away</show>
  <status>Online lagi nanti sore</status>
</presence>
```

Gambar 2. 9 Stanza Presence XMPP

Dalam aplikasi *instant messaging*, bagian *presence* biasanya dimunculkan pada *roster* (istilah untuk daftar kontak). *Roster* akan termuat daftar JabberID dan status dari para pengguna yang tersimpan didalamnya akan diberitahukan secara berkala (Saint-Andre et al., 2009).

2.10.10 Stanza IQ (Info/Query)

Stanza IQ menyediakan struktur untuk melakukan interaksi *request-response* dan alur kerja sederhana, mirip seperti metode *GET*, *POST*, dan *PUT* yang familiar pada HTTP. *Stanza IQ* hanya bisa memiliki satu *payload* tidak seperti *stanza message* yang dapat memiliki beberapa *payload* sekaligus. Dimana *payload* pada IQ mendefinisikan permintaan proses atau aksi yang harus dilakukan oleh penerima. Entiti yang mengirimkan *stanza IQ* harus selalu menerima tanggapan.

Permintaan dan tanggapan dapat dilacak menggunakan atribut id, yang dihasilkan melalui permintaan entitas dan akan dimasukkan pada tanggapan entitas. Atribut *type* pada *stanza IQ* ada beberapa macam:

1. *Get*

Atribut tipe permintaan untuk meminta suatu informasi.

2. *Set*

Atribut tipe permintaan untuk memberikan suatu informasi atau membuat permintaan.

3. *Result*

Atribut tipe tanggapan yang mengembalikan suatu permintaan, sesuai dengan yang meminta *GET* atau *SET*.

4. Error

Atribut tipe tanggapan atau entitas perantara untuk memberitahukan bahwa permintaan *GET* atau *SET* mengalami suatu kesalahan atau permintaan tidak dapat diproses karena suatu hal.

Contoh pengguna *stanza* IQ dalam kasus meminta *roster* (daftar kontak) dan menambahkan kontak baru ke dalam *roster*.

```
<iq
  from="person1@server.org"
  id="abcdefgh"
  type="get">
  <query xmlns="jabber:iq:roster"/>
</iq>
```

Gambar 2. 10 Stanza IQ Get Roster XMPP

```
<iq
  from="person1@server.org"
  id="abcdefgh"
  type="result">
  <query xmlns="jabber:iq:roster">
    <item jid="friend@server.org"/>
  </query>
</iq>
```

Gambar 2. 11 Stanza IQ Result Get Roster XMPP

Pada contoh diatas pengguna *person1@server.org* meminta daftar kontak yang dimiliki dengan cara memberitahukan melalui permintaan *stanza* IQ bertipe *GET* dan *query* berupa *jabber:iq:roster*.

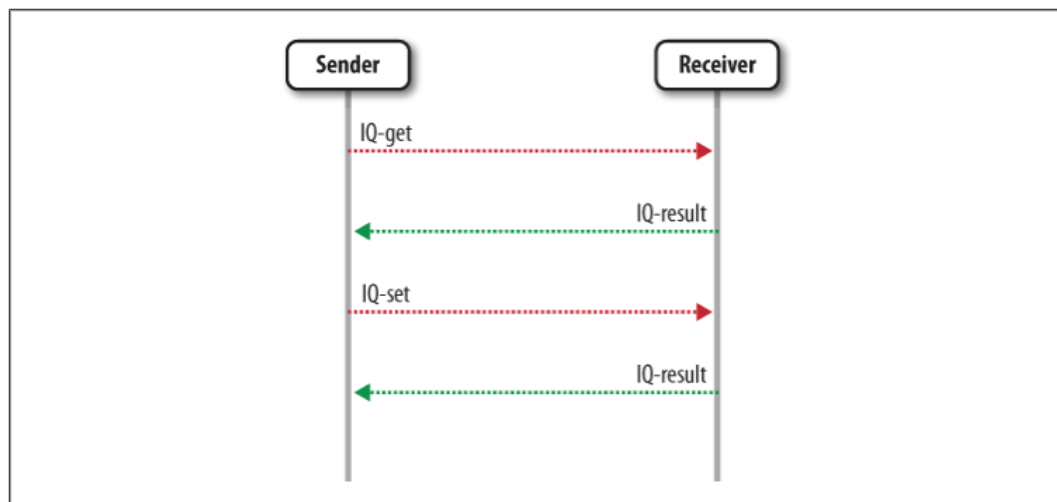
```
<iq
  from="person1@server.org"
  id="abcdefgh"
  type="set">
  <query xmlns="jabber:iq:roster">
    <item jid="newFriend@server.org"/>
  </query>
</iq>
```

Gambar 2. 12 Stanza IQ Set Roster XMPP

```
<iq
  from="person1@server.org"
  id="abcdefghH"
  type="result"/>
```

Gambar 2. 13 Stanza IQ Result Set Roster XMPP

Pada contoh yang disajikan pengguna *person1@server.org* meminta untuk menambahkan kontak atas nama *newFriend@server.org* dengan cara memberitahukan melalui permintaan *stanza* IQ bertipe *SET* dan *query* berupa *jabber:iq:roster* serta kontak baru tersebut dimasukkan kedalam *query* sebagai *payload* dengan tag berupa *<item>*. Penggunaan *stanza* IQ jika dilihat dalam gambar adalah sebagai berikut (Saint-Andre et al., 2009).



GAMBAR 2.4 Mekanisme Request Response Stanza IQ

2.11 Perbandingan Mengirim Data dengan XML dan JSON

Format data JSON memiliki keunggulan dibandingkan dengan format data XML diberbagai bentuk pengujian yang telah dilakukan (Breje et al., 2018). Hasil dari pengujiannya adalah sebagai berikut:

1. *GET Without GZIP Compression*

Table 2.2 Perbandingan JSON dan XML GET Without GZIP

Records No.	JSON Size	XML Size	JSON Second	XML Second
1000	281 KB	337 KB	0.49 s	1.03 s
5000	1402 KB	1689 KB	0.82 s	2.95 s
10000	2805 KB	3205 KB	1.63 s	5.1 s

Tabel hasil perbandingan permintaan data dalam format JSON dan XML tanpa dilakukan kompresi untuk jumlah data 1000, 5000, dan 10000.

2. *GET With GZIP Compression*

Table 2.3 Perbandingan JSON dan XML GET With GZIP

Records No.	JSON Size	XML Size	JSON Second	XML Second
1000	52.9 KB	73.4 KB	0.32 s	0.6 s
5000	263.1 KB	356.8 KB	0.61 s	2.21 s
10000	529.3 KB	731.3 KB	1.22 s	4.14 s

Tabel hasil perbandingan permintaan data dalam format JSON dan XML dengan dilakukan kompresi untuk jumlah data 1000, 5000, dan 10000.

3. *POST*

Table 2.4 Perbandingan JSON dan XML POST

Records No.	JSON Second	XML Second	% XML over JSON
1000	1.25 s	2.21 s	176
5000	5.09 s	8.78 s	173
10000	11.11 s	17.02 s	153

Tabel hasil perbandingan *request POST* data dalam format JSON dan XML untuk jumlah data 1000, 5000, dan 10000.

4. *PUT*

Table 2.5 Perbandingan JSON dan XML PUT

Records No.	JSON Second	XML Second	% XML over JSON
1000	1.9 s	2.73 s	143
5000	9.73 s	17.43 s	179
10000	18.9 s	25.44 s	134

Tabel hasil perbandingan *request PUT* data dalam format JSON dan XML untuk jumlah data 1000, 5000, dan 10000.

5. *DELETE*

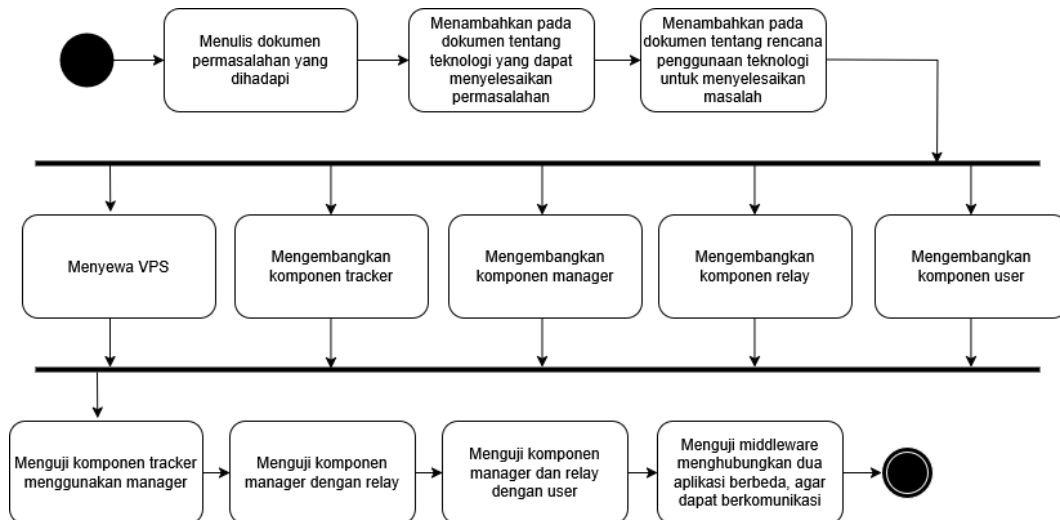
Table 2.6 Perbandingan JSON dan XML *DELETE*

Records No.	JSON Second	XML Second	% XML over JSON
1000	0.4 s	0.55 s	137
5000	0.49 s	0.58 s	118
10000	0.58 s	0.7 s	120

Tabel hasil perbandingan *request DELETE* data dalam format JSON dan XML untuk jumlah data 1000, 5000, dan 10000.

BAB III METODOLOGI PENELITIAN

3.1 Tahapan Penelitian

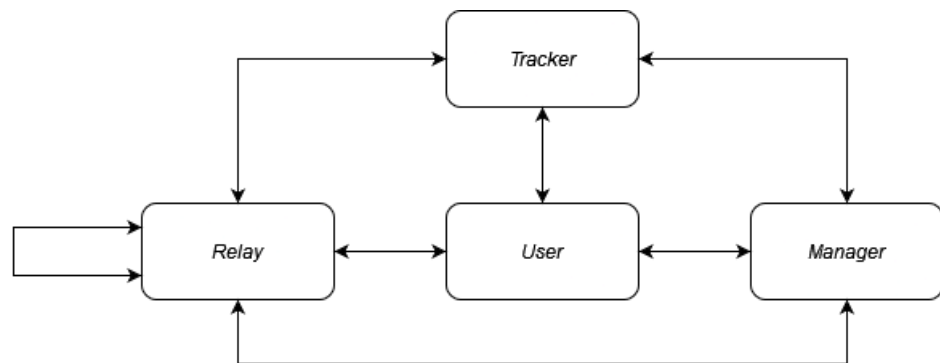


Gambar 3.1 Flowchart Tahapan Penelitian Pembuatan *Middleware* Komunikasi Menggunakan XMPP dan Arsitektur *microservice*

Penelitian akan dimulai dengan penulisan dokumen. Permasalahan pembuatan *middleware* komunikasi, dapat diselesaikan dengan mengimplementasikan protokol XMPP dengan bantuan *socket programming*. Rencana dari penelitian adalah mengubah protokol XMPP dari *monolith* menjadi *microservice*. Hal ini dilakukan agar kemampuan untuk menampung user yang ingin berkomunikasi dapat ditingkatkan. Kemampuan ini dapat direalisasikan karena komponen pada *microservice* saling berkaitan dan bukan menjadi satu sistem utuh.

Kemudian mencari penyedia VPS (*Virtual Private Server*). VPS akan menjadi tempat menjalankan program *tracker*, *manager*, dan *relay*. Program akan dibangun dalam bentuk *socket programming* dan arsitektur *microservice*. Setelah semuanya selesai dan dijalankan, *user* dapat melakukan koneksi *socket* TCP dengan sistem, dan melakukan pengujian untuk komponen *tracker*, *manager*, dan *relay*. Pengembangan dan pengujian penelitian dapat dilihat pada sub bab 3.6, tentang tahapan pengembangan.

3.2 Arsitektur *Middleware* Komunikasi dengan XMPP



Gambar 3.2 Arsitektur *Microservice Middleware* Komunikasi dengan XMPP

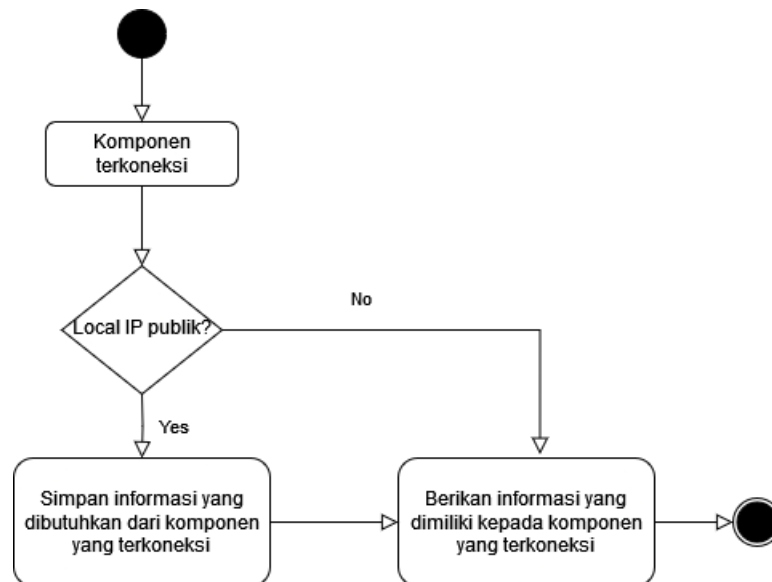
Arsitektur akan dibuat sebagai *microservice*, dengan setiap *service* akan dibedakan berdasarkan komponen utama. Arsitektur akan memiliki tiga komponen utama yaitu *tracker*, *manager*, dan *relay* dan satu komponen penguji yaitu *user*. Adapun peran dari masing-masing komponen adalah sebagai berikut:

1. **Tracker:** Memiliki peran sebagai tempat pertukaran informasi IP yang dimiliki oleh masing-masing komponen.
2. **Manager:** Memiliki fungsi untuk manajerial seperti pengaturan keseimbangan jaringan pada *relay*, pengelolaan aktivitas *user* seperti autentikasi, kontak, dan status.
3. **Relay:** Memiliki peran untuk menjembatani data yang berasal dari satu *user* ke *user* yang lain. Komponen *user* akan memiliki IP *Private*.
4. **User:** Memiliki peran sebagai klien dari jaringan yang telah dibuat oleh *tracker*, *manager*, dan *relay*.

Arsitektur akan dibangun dengan jaringan terkoneksi menggunakan *socket* bertipe TCP (*Transmission Control Protocol*) *long-lived connections*. Sistem *socket* tipe TCP menggunakan teknik koneksi (*connection*) antara dua *socket client*. Format protokol untuk *socket* TCP adalah (*socket_protocol*, *client_1_address*, *client_1_port*, *client_2_address*, *client_2_port*).

3.2.1 Komponen *Tracker*

Komponen *tracker* akan memiliki fungsionalitas sebagai pengelola alamat IP publik dari komponen dan status dari komponen. Komponen yang akan dicatat alamat IP publiknya adalah *manager*, dan *relay*.

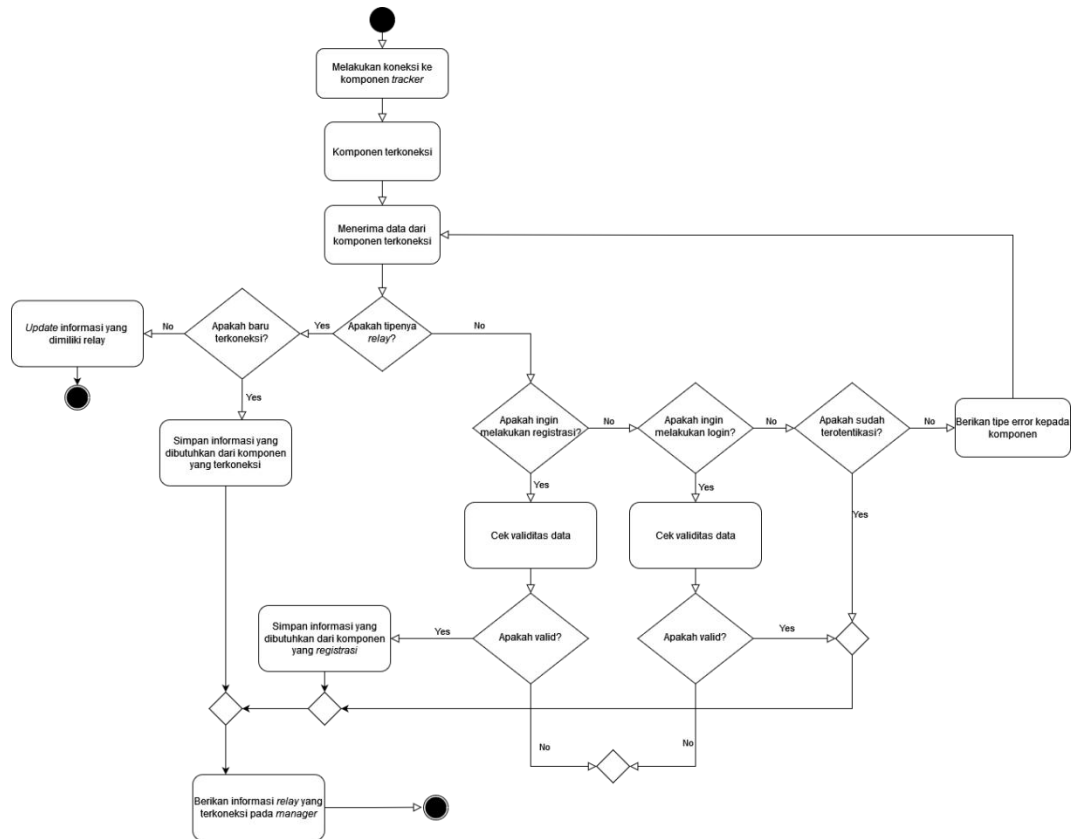


Gambar 3.3 Activity Diagram Komponen *Tracker*

3.2.2 Komponen *Manager*

Komponen *manager* adalah komponen manajerial. Komponen *manager* akan memiliki fungsionalitas sebagai berikut.

1. Pengelolaan autentikasi *user*
2. Pengelolaan *roster user*
3. Pengelolaan *presence user*.
4. Pengelolaan keseimbangan koneksi pada *relay*.

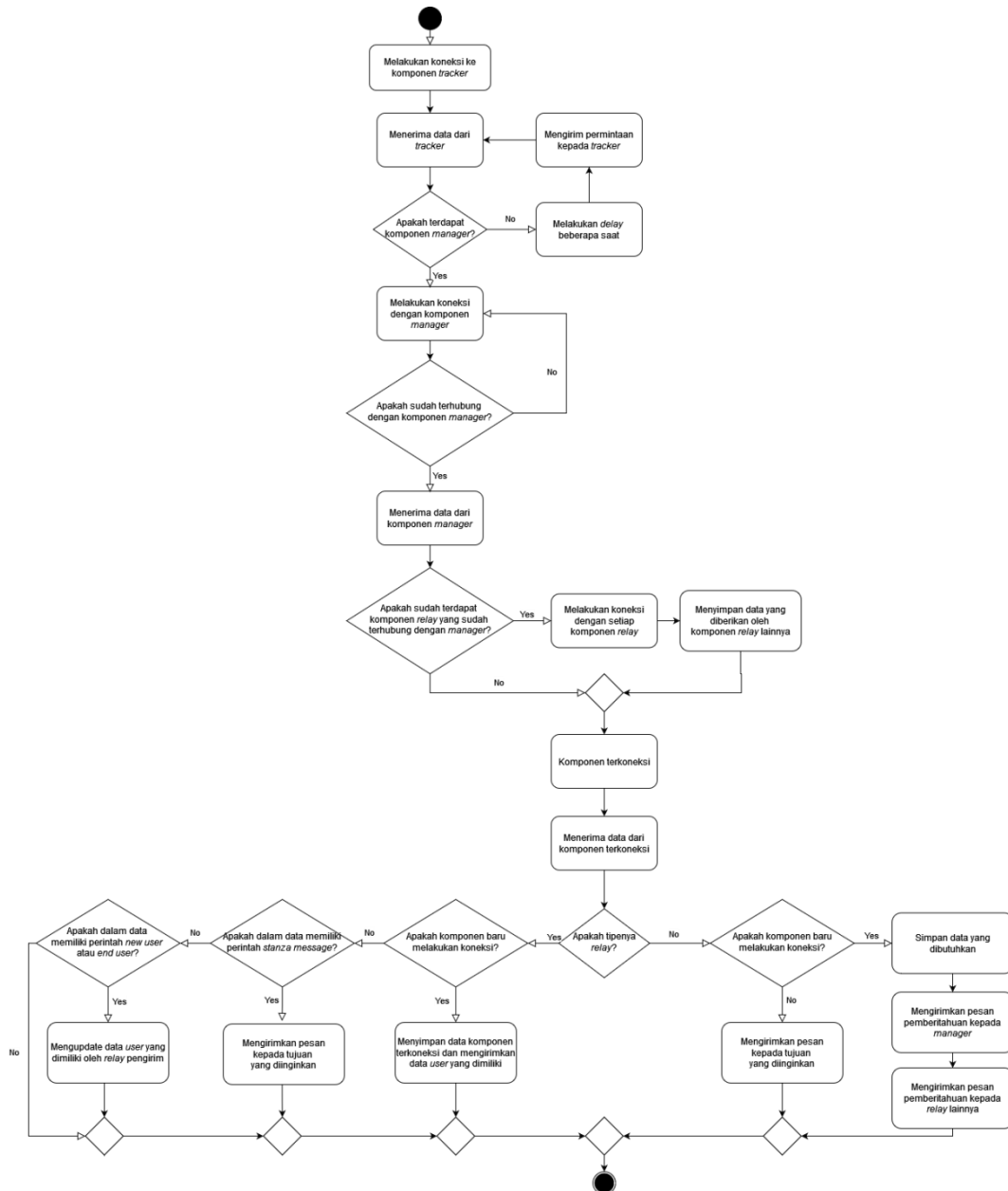


Gambar 3.4 Activity Diagram Komponen Manager

3.2.3 Komponen Relay

Komponen *relay* sebagai komponen pada *middleware* komunikasi, yang memiliki fungsionalitas sebagai berikut.

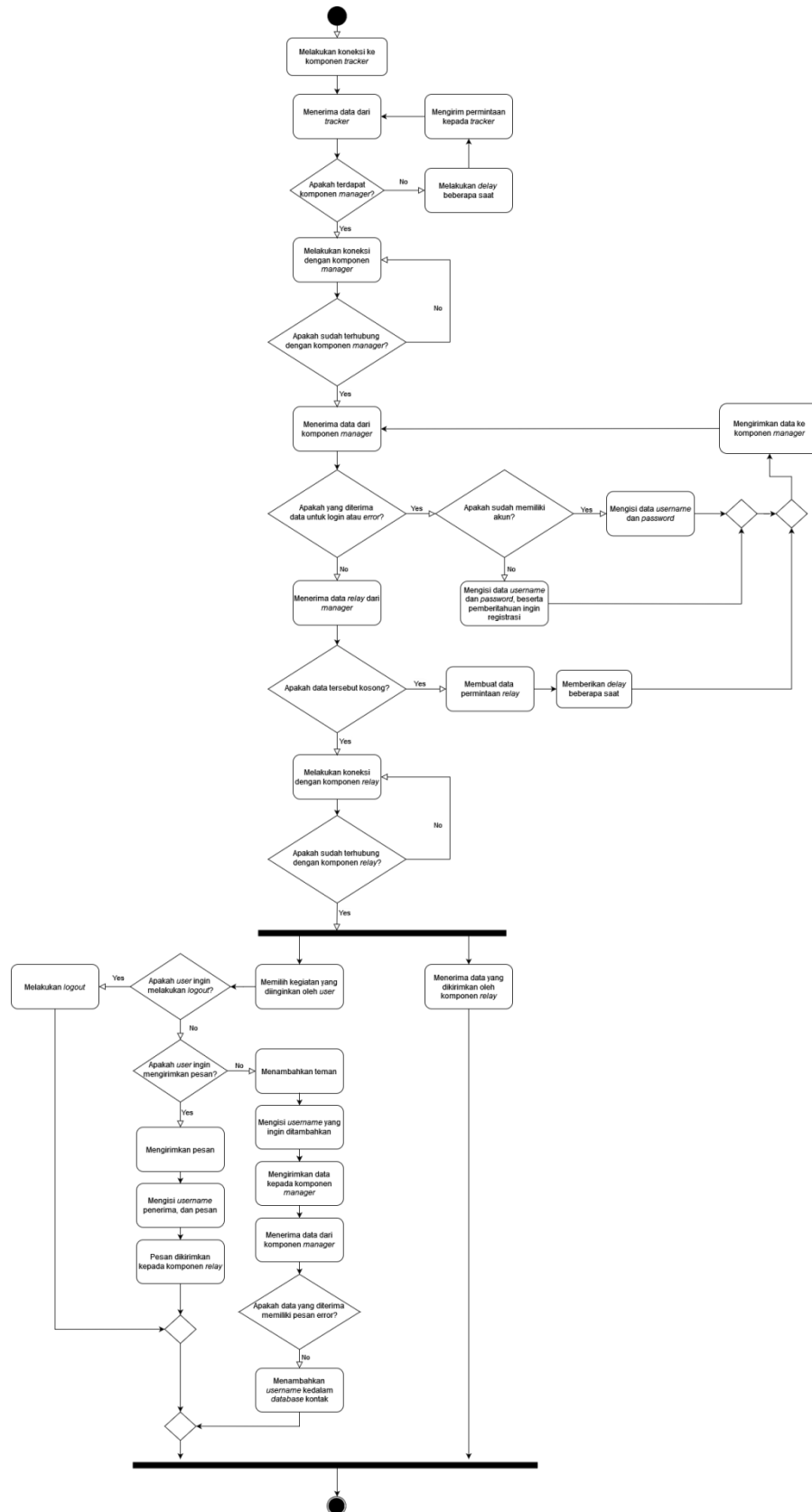
1. Pengelolaan koneksi sesama *relay*.
2. Pengelolaan *user* yang terkait dengan dirinya ataupun *relay* lainnya.
3. Pengelolaan stanza message sebagai format pertukaran data dari satu *user* ke *user* lainnya.



Gambar 3.5 Activity Diagram Komponen Relay

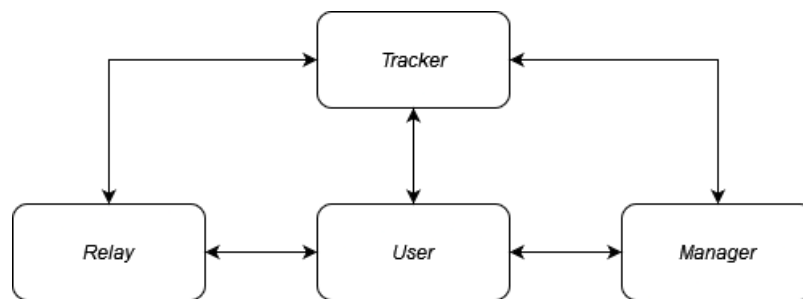
3.2.4 Komponen User

Komponen *user* memiliki peran sebagai penguji sistem yang telah dibuat. Mulai dari pemberian IP oleh *tracker*, manajerial dari komponen *manager*, dan kemampuan mengirimkan pesan yang ditangani oleh *relay*.



Gambar 3.6 Activity Diagram Komponen User

3.2.5 Relasi *Tracker* dengan Komponen Lainnya



Gambar 3.7 Relasi Antara *Tracker* dengan Komponen Lainnya

Peran dari *tracker* adalah untuk mengumpulkan alamat IP setiap koneksi dari *manager* dan *relay* yang terkoneksi ke dirinya dan memberikan setiap alamat IP yang sudah dicatat oleh dirinya kepada komponen yang terkoneksi ke dirinya.

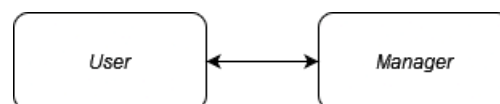
3.2.6 Relasi *Relay* dengan *Manager*



Gambar 3.8 Relasi Antara *Relay* dan *Manager*

Peran *manager* terhadap *relay* adalah sebagai penjaga kestabilan *relay* agar pekerjaan *relay* tidak berat. Komponen *manager* akan mencatat IP dari *relay* dan akan mengatur apabila ada *user* yang ingin bergabung ke dalam sistem *chat*. Komponen *manager* mengatur dengan cara mencari *relay* yang memiliki beban paling sedikit apabila *relay* terdapat lebih dari satu.

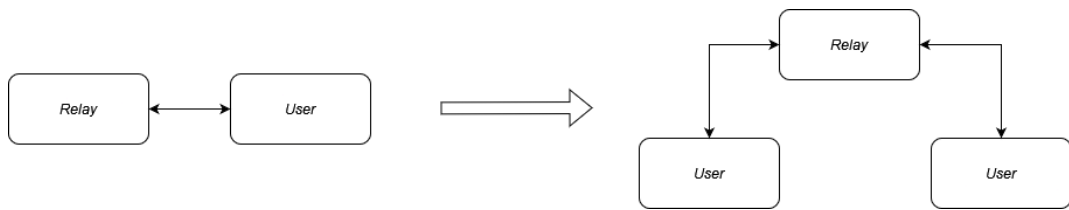
3.2.7 Relasi *User* dengan *Manager*



Gambar 3.9 Relasi Antara *User* dengan *Manager*

Peran *manager* terhadap *user* adalah sebagai pemberi alamat *relay* dengan koneksi paling rendah dan pelayanan untuk autentikasi, pengelolaan *roster*, dan *presence*.

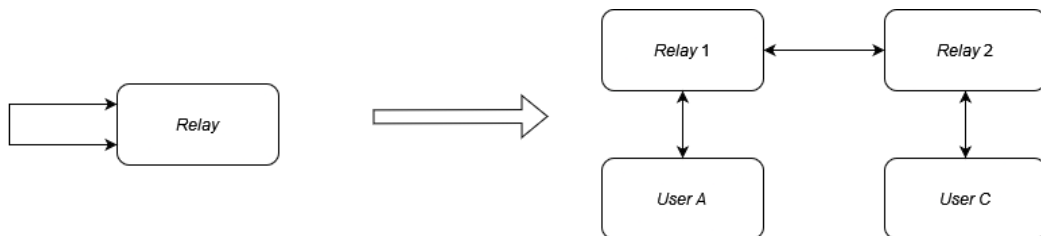
3.2.8 Relasi *User* dengan *Relay*



Gambar 3.10 Relasi Antara *Relay* dan *User*

Peran *relay* terhadap *user* adalah sebagai jembatan pertukaran data pesan antar *user*.

3.2.9 Relasi Antar *Relay*



Gambar 3.11 Relasi Antar *Relay*

Relasi antar *relay* berfungsi sebagai jembatan data untuk pesan *user* yang mana *user* pengirim dan tujuan memiliki koneksi *relay* yang berbeda.

3.3 Format XMPP

Format dari protokol XMPP adalah XML. Pada penelitian ini format XML akan diganti dengan format JSON. Contoh pengubahan format XML menjadi JSON akan terlihat seperti berikut (contoh menggunakan XMPP *stanza message*):

```
<message
  from="person@server.org"
  to="friend@server.org"
  type="chat">
  <body>Hallo teman!</body>
</message>
```

Gambar 3. 12 *Stanza Message* XMPP

```

{
  "stanza": "message",
  "from": "person@server.org",
  "to": "friend@server.org",
  "type": "chat",
  "body": "Hallo teman!"
}

```

Gambar 3. 13 Stanza Message JSON

3.4 Penyimpanan Data

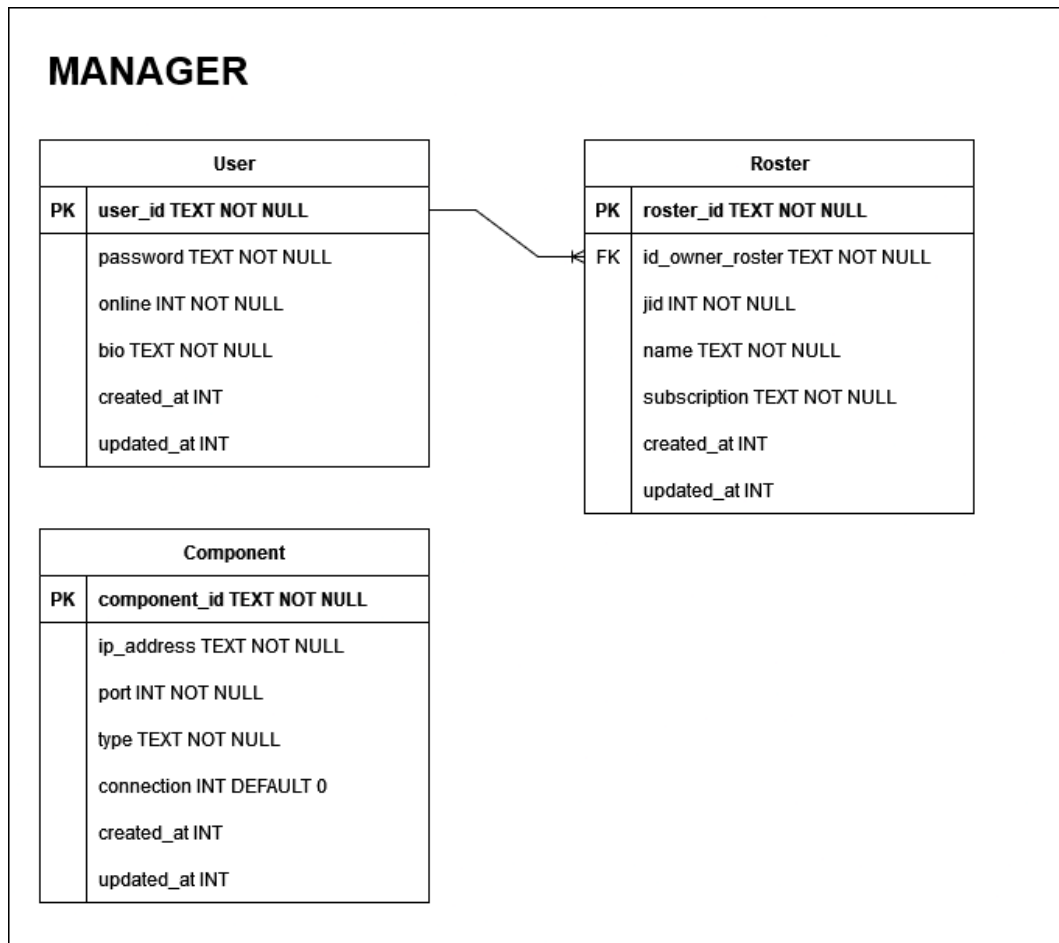
Setiap komponen memiliki data yang akan disimpan, oleh sebab itu komponen memerlukan struktur penyimpanan data untuk data yang ingin disimpan olehnya. Penyimpanan data akan dibentuk menggunakan teknologi SQL (*Structured Query Language*).

3.4.1 Struktur Penyimpanan Data Tracker

TRACKER	
Components	
PK	<u>component_id</u> int NOT NULL
	ip_address string NOT NULL
	port int NOT NULL
	type string NOT NULL
	created_at date NULL

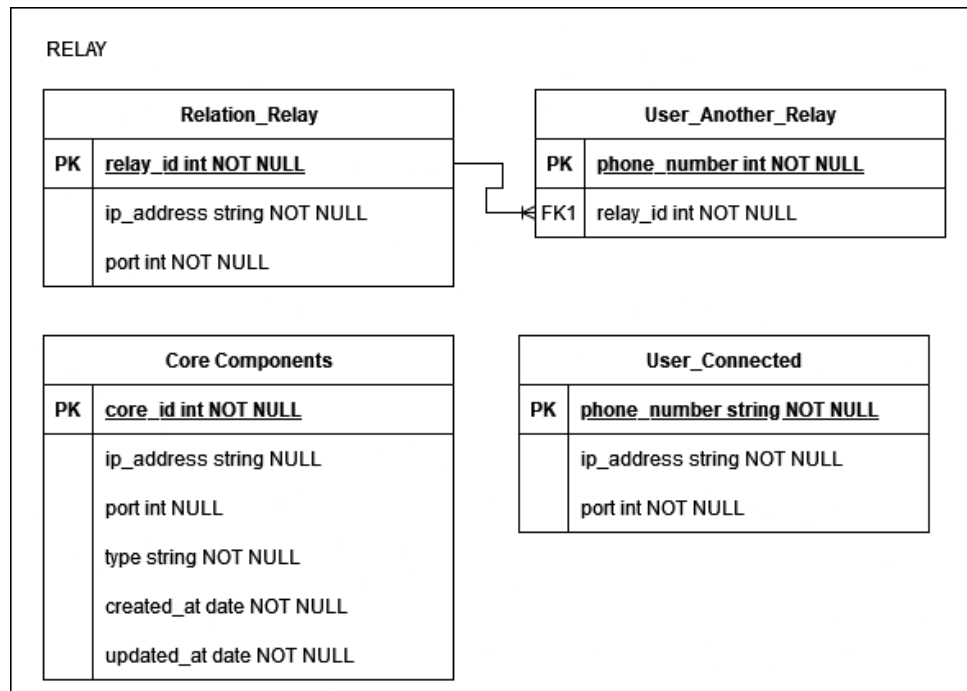
Gambar 3.14 Database Pada Komponen Tracker

3.4.2 Struktur Penyimpanan Data *Manager*



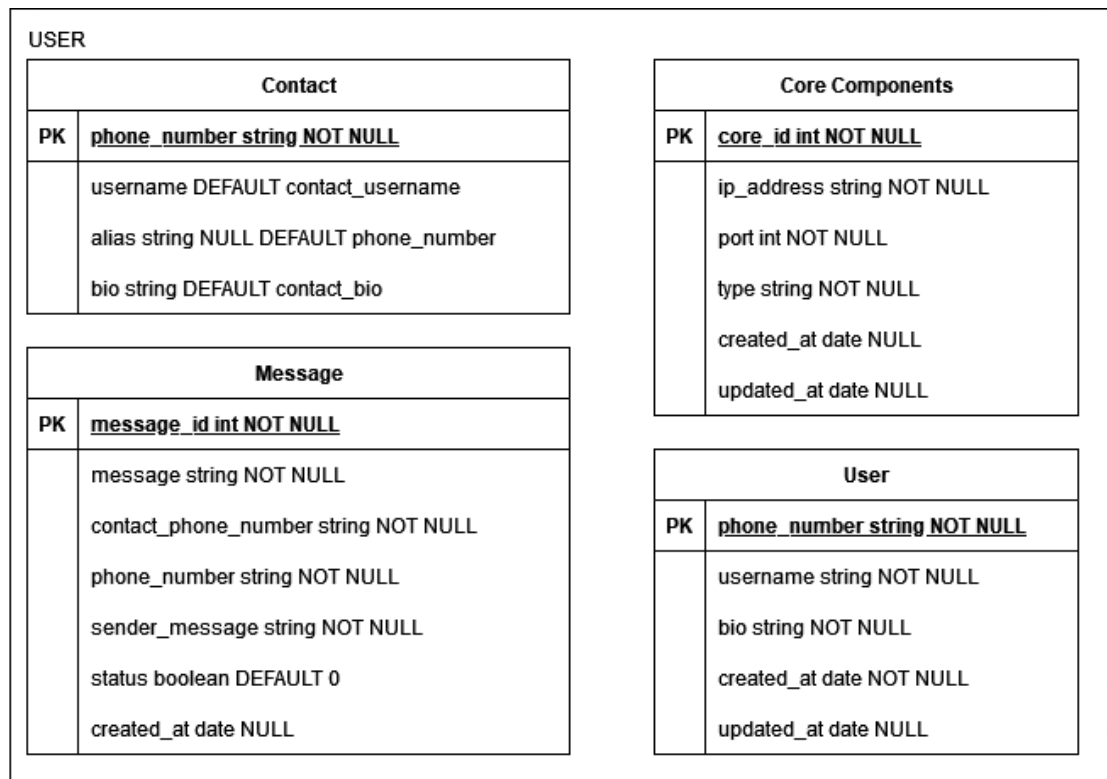
Gambar 3.15 Database Pada Komponen *Manager*

3.4.3 Struktur Penyimpanan Data Relay



Gambar 3.16 Database Pada Komponen Relay

3.4.4 Struktur Penyimpanan Data User



Gambar 3.17 Database Pada Komponen User

3.5 Alat dan Bahan Penelitian

Penelitian ini menggunakan beberapa alat untuk membuat sistem, mengelola data, dan menjalankan sistem yaitu:

1. VSCode.
2. SQLite3.
3. Python 3.
4. VPS (*Virtual Private Server*) sebanyak dua.
5. Perangkat komputer/laptop.

3.6 Tahapan Pengembangan

3.6.1 Pemilihan VPS

VPS dipilih berdasarkan sumber daya yang ditawarkan beserta harga yang paling sesuai. Pada penelitian ini VPS yang dipilih adalah VPS dengan sumber daya yang tidak terlalu besar dan harga yang relatif murah karena pada penelitian ini hanya akan diujikan dalam ruang lingkup yang kecil dan sederhana.

3.6.2 Pengembangan Komponen

Komponen akan dikembangkan secara bertahap, diawali dengan membuat dasar tiap-tiap komponen dan memastikan tiap komponen dapat saling terhubung, mengembangkan *tracker* agar dapat melakukan fungsinya dengan baik, mengembangkan *manager* mulai dari autentikasi, pengelolaan kontak dan status, mengembangkan *relay* untuk jalur pertukaran pesan komponen *user*, dan mengembangkan *user* untuk menguji apakah pengembangan semuanya sudah sesuai.

3.7 Skema Uji

Skema untuk menguji sistem penghubung komunikasi dengan XMPP, agar dapat berjalan dengan baik dan benar pada penelitian adalah sebagai berikut:

1. Menjalankan program *tracker* pada VPS pertama.
2. Menjalankan program *manager* pada VPS kedua.
3. Menguji fungsionalitas menerima koneksi dan penyimpanan komponen pada *tracker*.
4. Menjalankan program *relay* pada VPS kedua.

5. Menguji fungsionalitas memberikan komponen yang disimpan oleh *tracker* kepada komponen yang baru terkoneksi.
6. Menguji fungsionalitas menerima koneksi dan penyimpanan komponen pada *manager*.
7. Menjalankan program *relay* pada VPS pertama.
8. Menguji fungsionalitas menerima koneksi dan memberikan data konfigurasi awal pada *relay* yang berada pada VPS kedua.
9. Menjalankan program *user* pada beberapa perangkat komputer/laptop yang memiliki LAN berbeda.
10. Menguji setiap fungsionalitas pada *manager*, diantaranya adalah:
 - a. Registrasi
 - b. *Login*
 - c. Mendapatkan komponen *relay* dengan koneksi paling sedikit.
 - d. Pengelolaan *roster*.
 - e. Pengelolaan *presence*.
11. Menguji fungsionalitas *relay* sebagai pengirim data antar *user*.

3.7.1 Metrik Pengujian

Performa sistem penghubung komunikasi dengan XMPP akan dinilai dengan menggunakan metrik sebagai berikut:

1. **Proses Komunikasi:** komunikasi antar komponen dapat berjalan dengan baik dan benar.
2. **Konsistensi Data:** kemampuan sistem untuk mengirimkan data sehingga tidak akan ada data yang hilang selama proses pengiriman data berlangsung.
3. **Fungsionalitas:** kemampuan setiap komponen dalam menjalankan tugas dan perannya masing-masing.

3.7.2 Rancangan Pengujian

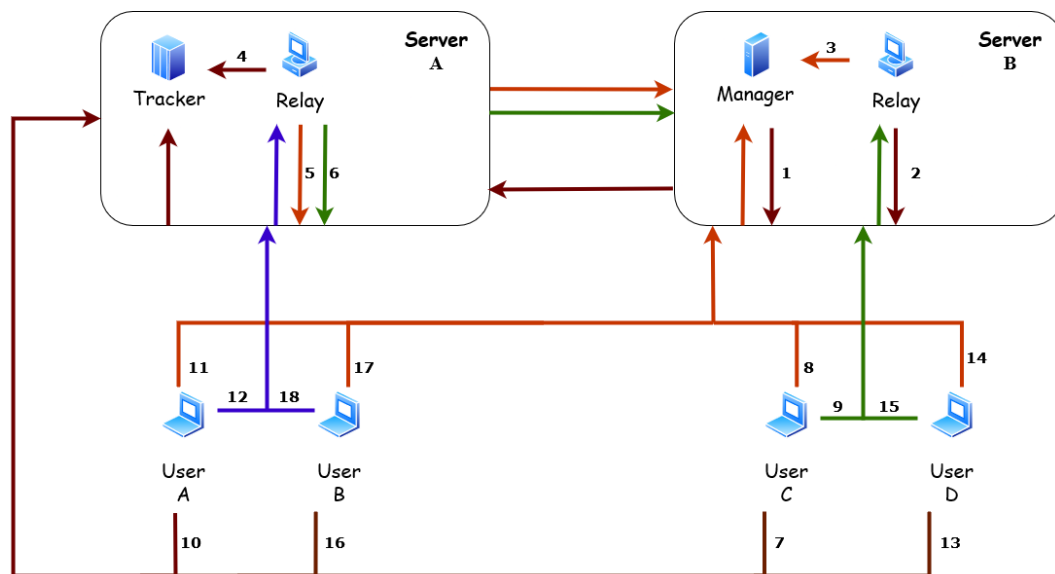
Pengujian akan dirancang menjadi dua buah VPS/server, VPS pertama akan menjalankan program *tracker* dan *relay*, dan VPS kedua akan menjalankan program *manager* dan *relay*. Komponen penguji akan dijalankan pada komputer/laptop yang memiliki LAN berbeda.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Implementasi Lingkungan Program dan Sistem

Lingkungan program dibuat menjadi dua yaitu, lingkungan program komponen *tracker* dan *relay*, dan komponen *manager* dan *relay*. Satu lingkungan program dibuat pada satu VPS atau *server*. IP yang dimiliki VPS atau *server* adalah IP publik dan dapat dilakukan koneksi *socket TCP* kepada dirinya.



Gambar 4. 1 Implementasi Relasi Antar Komponen

Keterangan dari implementasi sistem:

1. Panah keluar dari suatu komponen menandakan bahwa komponen tersebut akan melakukan koneksi kepada komponen lainnya.
2. Komponen target diwakili dengan warna sebagai berikut:
 - Coklat adalah komponen tracker.
 - Jingga adalah komponen manager.
 - Hijau adalah komponen relay pada server B.
 - Biru adalah komponen relay pada server A.
3. Urutan angka yang berada disekeliling komponen, menandakan urutan mereka akan melakukan koneksi, dimulai dari yang terkecil ke terbesar.

Sistem akan dimulai dari menyalakan *tracker*, kemudian *manager*, dan terakhir *relay*. Ketika ketiga komponen ini sudah tersedia, maka *user* yang melakukan koneksi ke sistem akan dapat menggunakan sistem untuk berkomunikasi. Sistem memiliki empat jenis folder utama, yaitu:

1. *Tracker*, berisi kebutuhan komponen agar menjadi komponen *tracker*.
2. *Manager*, berisi kebutuhan komponen agar menjadi komponen *manager*.
3. *Relay*, berisi kebutuhan komponen agar menjadi komponen *relay*.
4. *Utils*, berisi kebutuhan utilitas yang secara umum dibutuhkan oleh masing-masing komponen.

Sistem memiliki dua jenis *file* yaitu, *file* berisi tentang kelas dan *file* berisi fungsionalitas. Adapun *file* berisi tentang kelas yaitu *init_db.py* terdapat dalam database, *socketServer.py* terdapat dalam folder *utils\kelas*, dan *socketClient.py*. Seluruh kodenya dapat dilihat di <https://github.com/Jarsters/socket-xmpp-json>.

4.2 Utilitas Umum Komponen

4.2.1 Kelas *SocketServer*

```
import socket

class SocketServer:
    def __init__(self, port=5000):
        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        try:
            self.socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        except:
            self.socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEPORT, 1)
        self.localIP = self.getMyLocalIP()
        self.socket.bind((self.localIP, port))
        self.socket.listen()
    def getMyLocalIP(self):
        udp = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        udp.connect(("8.8.8.8", 80))
        output = udp.getsockname()[0]
        udp.close()
        del udp
        return output
```

Gambar 4. 2 Kelas untuk Menjadikan Komponen Sebagai *Server*

Kelas *SocketServer* adalah sebuah kelas yang menyediakan kemampuan untuk komponen menjadi *server*. Kelas *SocketServer* menggunakan koneksi bertipe TCP dan jika tidak menentukan *port* saat memanggil kelas ini, maka *port*

akan otomatis ditetapkan ke *port* 5000. Metode *getMyLocalIP* adalah metode untuk mendapatkan IP lokal dari komponen yang memanggil metode tersebut.

4.2.2 Utilitas *Get_Time*

```
import time, datetime

def get_timestamp():
    return int(time.time() * 1000)

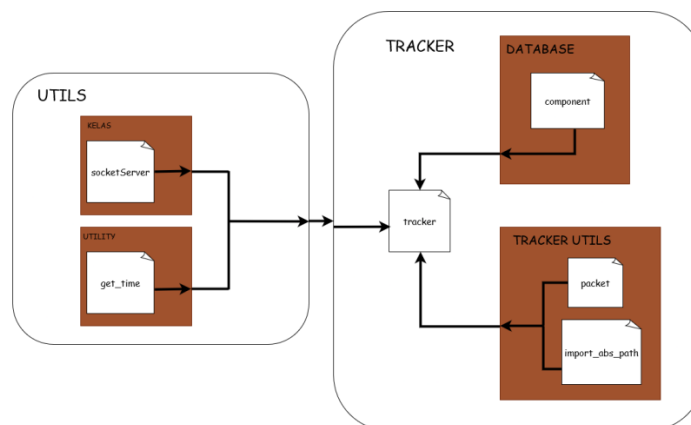
def see_in_date(timestamp):
    return datetime.datetime.fromtimestamp(timestamp/1000).strftime('%Y-%m-%d %H:%M:%S')
```

Gambar 4. 3 Utilitas untuk Mengelola Waktu Menjadi *Timestamp*

Utilitas ini membantu dalam penyediaan *timestamp* yang akan digunakan untuk menyimpan data pada *database* komponen.

4.3 Implementasi Komponen *Tracker*

4.3.1 Diagram Komponen *Tracker*



Gambar 4. 4 Diagram Lengkap Komponen *Tracker*

Komponen *tracker* memiliki file utama yaitu *tracker.py* yang terdapat dalam folder *tracker*. Didukung oleh *database* yang akan mengelola komponen, terletak pada folder *tracker/database* dengan nama file *component.py*. Serta memiliki kemampuan untuk mengelola pesan keluar masuk pada jaringan, dan dapat meng-*import* file yang tidak berada pada lingkungannya dengan bantuan file *import_abs_path*. Sehingga *tracker* dapat memiliki kemampuan untuk menjadikan dirinya sebagai *server*, dan mengelola *timestamp*.

4.3.2 Cara Kerja Tracker

```
import socket
import json
import threading

from tracker_utils.import_abs_path import import_outside_utils
from tracker_utils.packet import get_message, send_message

from database.component import delete_component_by_id, delete_components, get_all, get_components, save_component_to_db
try:
    module_socket = import_outside_utils("utils\\kelas\\", "socketServer.py")
except:
    print("Masuk except 1")
    module_socket = import_outside_utils("utils/kelas/", "socketServer.py")
try:
    module_get_time = import_outside_utils("utils\\utility\\", "get_time.py")
except:
    print("Masuk except 2")
    module_get_time = import_outside_utils("utils/utility/", "get_time.py")
SocketServer = module_socket.SocketServer
get_timestamp = module_get_time.get_timestamp
```

Gambar 4. 5 Tracker Import Starter

Komponen *tracker* akan meng-*import* setiap hal yang dibutuhkan. Modul *socket* untuk pengelolaan koneksi. Modul *json* untuk pengelolaan data json. Modul *threading* untuk pengelolaan jaringan yang akan berjalan berbarengan. Modul *import_abs_path* untuk meng-*import* file yang bukan dari lingkungan *tracker*. Modul *packet* untuk pengelolaan data keluar masuk pada jaringan. Modul *database* untuk pengelolaan data. Modul *socketServer* untuk membuat *tracker* memiliki kemampuan menjadi *server*. Modul *get_time* untuk pengelolaan waktu menjadi *timestamp*.

```
s = SocketServer()
server = s.socket
print(f"Tracker listening .... on ({s.localIP} - 5000)")
print(f"Daftar koneksi dalam sistem {get_components()}")
print("=====")

while True:
    connection, address = server.accept()
    messages = get_message(connection)
    id_component = None
    for message in messages:
        message = json.loads(message)
        print(f"Koneksi baru: {message}\r\n")
        ip_is_private = message.get("is_private")
        tipe = message.get('type').lower()
        if(not ip_is_private or tipe == 'manager' or tipe == 'relay'):
            data = list(message.values())
            id_component = save_component_to_db(data, get_timestamp)
        print(f"Daftar koneksi dalam sistem {get_all()}\r\n")
        send_connected_components_to_component(connection)
        threading.Thread(target=handleComponent, args=(connection, message, id_component), daemon=True).start()
    print("=====")
```

Gambar 4. 6 Tracker Accept Connection

Komponen *tracker* akan membuat dirinya menjadi *server*. Kemudian akan menunggu koneksi yang masuk kepada dirinya. Ketika mendapatkan pesan dari koneksi tersebut bertipe *manager* atau *relay*, maka alamat IP dari koneksi tersebut

akan dicatat. Komponen yang terkoneksi akan diberikan seluruh alamat IP komponen lainnya yang memiliki status aktif. Komponen tersebut kemudian akan dikelola dengan *threading* yang memiliki fungsi *handleComponent*.

```
root@ift2:~/socket-xmpp-json/tracker# python3 tracker.py
Tracker listening .... on (103.178.153.189 - 5000)
Daftar koneksi dalam sistem []
=====
Koneksi baru: {'ip_local': '103.166.156.127', 'port': 39573, 'type': 'manager', 'is_private': False}
Daftar koneksi dalam sistem [{'ip_local': '103.166.156.127', 'port': 39573, 'type': 'manager', 'is_private': '0', 'status': 1}]
Koneksi yang diberikan kepada komponen terkoneksi [{'ip_local': '103.166.156.127', 'port': 39573, 'type': 'manager', 'is_private': '0'}]
=====
Koneksi baru: {'ip_local': '103.178.153.189', 'port': 33496, 'type': 'relay', 'is_private': False}
Daftar koneksi dalam sistem [{'ip_local': '103.166.156.127', 'port': 39573, 'type': 'manager', 'is_private': '0', 'status': 1}, {'ip_local': '103.178.153.189', 'port': 33496, 'type': 'relay', 'is_private': '0', 'status': 1}]
Koneksi yang diberikan kepada komponen terkoneksi [{'ip_local': '103.166.156.127', 'port': 39573, 'type': 'manager', 'is_private': '0'}, {'ip_local': '103.178.153.189', 'port': 33496, 'type': 'relay', 'is_private': '0'}]
=====
Terjadi putus koneksi dengan id_component 2
Masuk ke delete component
{"error_msg": true, "type": "socket peer is closed"}
Daftar koneksi dalam sistem [{'ip_local': '103.166.156.127', 'port': 39573, 'type': 'manager', 'is_private': '0', 'status': 1}, {'ip_local': '103.178.153.189', 'port': 33496, 'type': 'relay', 'is_private': '0', 'status': 0}]
=====
Koneksi baru: {'ip_local': '103.178.153.189', 'port': 45873, 'type': 'relay', 'is_private': False}
Daftar koneksi dalam sistem [{'ip_local': '103.166.156.127', 'port': 39573, 'type': 'manager', 'is_private': '0', 'status': 1}, {'ip_local': '103.178.153.189', 'port': 33496, 'type': 'relay', 'is_private': '0', 'status': 0}, {'ip_local': '103.178.153.189', 'port': 45873, 'type': 'relay', 'is_private': '0', 'status': 1}]
Koneksi yang diberikan kepada komponen terkoneksi [{'ip_local': '103.166.156.127', 'port': 39573, 'type': 'manager', 'is_private': '0'}, {'ip_local': '103.178.153.189', 'port': 45873, 'type': 'relay', 'is_private': '0'}]
=====
```

Gambar 4. 7 Hasil Tracker Accept Connection

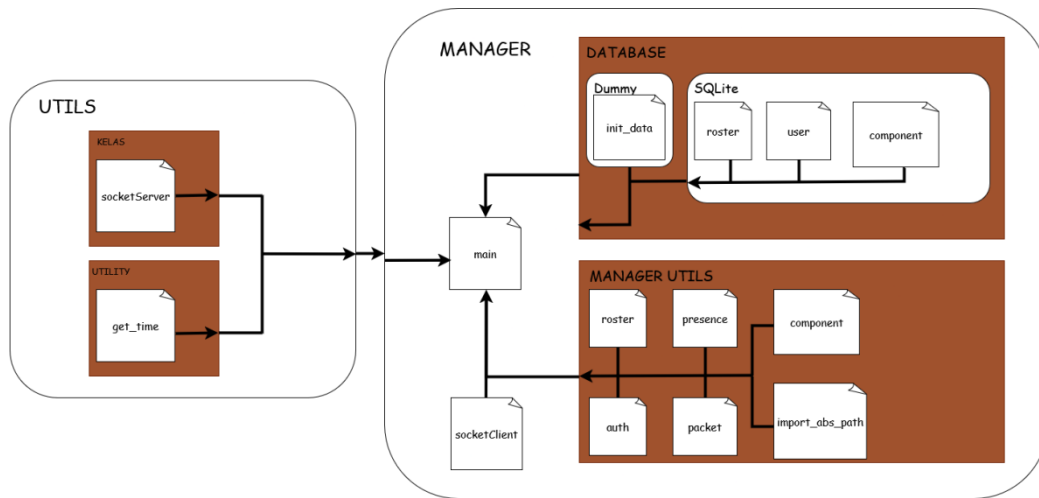
```
def handleComponent(communicate:socket.socket, msg, id_component):
    error = False
    while not error:
        try:
            messages = get_message(communicate)
            for msg in messages:
                message = json.loads(msg)
                if(message.get("message") and message.get("message").lower() == "get components"):
                    send_connected_components_to_component(communicate)
                elif(message.get("error_msg")):
                    print(f"Terjadi putus koneksi dengan id_component {id_component}")
                    delete_component(msg, id_component)
                    error = True
            except ConnectionAbortedError:
                print("Connection end 1 ...")
                delete_component(msg, id_component)
                break
            except ConnectionRefusedError:
                print("Connection end 2 ...")
                delete_component(msg, id_component)
                break
            except ConnectionError:
                print("Connection end 3 ...")
                delete_component(msg, id_component)
                break
            except Exception as e:
                print(e)
                print("Connection end 4 ...")
                delete_component(msg, id_component)
                break
```

Gambar 4. 8 Tracker Fungsi Menangani Komponen yang Terkoneksi

Fungsi *handleComponent* adalah untuk menyediakan kemampuan pada *tracker* apabila ada komponen yang terkoneksi meminta data komponen aktif terbaru.

4.4 Implementasi Komponen *Manager*

4.4.1 Diagram Komponen *Manager*



Gambar 4. 9 Diagram Komponen *Manager*

Komponen *manager* memiliki *file* utama yang terdapat dalam *folder manager* dengan nama *file* adalah *main.py*. Memiliki *file* *socketClient* yang memberikan kemampuan *manager* untuk menjadi *client* suatu komponen. Memiliki *file* pengelolaan autentikasi, *roster*, *presence*, *packet*, *component* *relay*, dan *import* *file* yang bukan dari lingkungannya, yang terdapat pada *folder manager/manager_utils*. Serta memiliki *file* pengelolaan *database* *init_data*, *roster*, *user*, dan *component* *relay*, yang terdapat pada *folder manager/database*.

4.4.2 Inisiasi Manager

```
import json
import threading
from string import printable
from random import randint
import ipaddress
import dotenv
import os

from manager_utils.presence import get_presence_by_jid, init_presence, logout, send_presence_to_someone, set_my_bio
from manager_utils.roster import delete_roster, get_packet_subscribed_for_init_entity, get_packet_unsubscribed_for_init_entity, get_rosters, set_roster
from manager_utils.auth import handle_auth
from manager_utils.packet import get_message, get_message_client, get_message_relay, get_message_tracker, send_message
from manager_utils.import_abs_path import import_outside_utils

from database.dummy.init_data import socket_user
from database.sqlite.component import components as c_db, convert_components_db, delete_all_data_component_db, \
    delete_components_db_by_id, save_component_to_db, get_relay_with_less_connection_db, update_total_connection
from database.sqlite.user import get_user_by_username, users as u_db, get_user_online
from database.sqlite.roster import get_roster_user, rosters as r_db

from socketClient import SocketClientManager

try:
    module_socket = import_outside_utils("utils\\kelas\\", "socketServer.py")
except:
    print("Masuk except 1")
    module_socket = import_outside_utils("utils/kelas/", "socketServer.py")
try:
    module_get_time = import_outside_utils("utils\\utility\\", "get_time.py")
except:
    print("Masuk except 2")
    module_get_time = import_outside_utils("utils/utility/", "get_time.py")
SocketServer = module_socket.SocketServer
get_timestamp = module_get_time.get_timestamp
```

Gambar 4. 10 *Manager Import Starter*

Manager melakukan import terhadap semua modul yang dibutuhkan.

```
ct, my_address = connect_to_tracker()
m = SocketServer(my_address[1])
manager = m.socket

print("Manager listening...")
```

Gambar 4. 11 *Manager Koneksi ke Tracker dan Menjadi Server*

Manager melakukan koneksi kepada *tracker* kemudian setelah berhasil *manager* akan menjadi *server* yang siap menerima koneksi komponen lain.

```
[root@jft manager]# python3 main.py
Local (y/n)?
Try to connecting.... 103.178.153.189 - 5000 (Tracker)
Connected
[{'ip_local': '103.166.156.127', 'port': 50250, 'type': 'manager', 'is_private': '0'}]
Manager listening...
```

Gambar 4. 12 *Hasil Koneksi ke Tracker dan Menjadi Server*

```

# Fungsionalitas untuk menerima setiap koneksi yang baru terhubung dengan manager
while True:
    try:
        connection, address = manager.accept()
        messages = get_message(connection)
        for msg in messages:
            message = json.loads(msg)
            print("=====")
            print(message)
            username_relay = None
            if(message.get("type").lower() == "relay"):
                print("KONFIGURASI RELAY")
                username_relay = random_username_for_relay(c_db, message)
                if(not username_relay):
                    obj_error = {
                        "error_msg": True,
                        "msg": "IP address dan port sudah terdaftar",
                        "error-type": "bad request"
                    }
                    send_message(connection, obj_error)
                    continue
                # Konfigurasi awal relay
                config_new_relay(message, username_relay)
                # Mengambil semua data komponen yang terkoneksi dengan manager
                c = get_all_components(c_db.get_all())
                print(f"Komponen yang diberikan kepada relay {username_relay} adalah:\n {c}")
                # Membuat packet yang akan dikirimkan kepada relay yang baru terkoneksi
                objek = {
                    "components": c,
                    "username": username_relay
                }
                # Mengirimkan semua data komponen kepada relay yang baru terkoneksi
                send_message(connection, objek)
            # Mendapatkan tipe dari komponen yang terhubung
            tipe = message.get('type')
            threading.Thread(target=handle_component, args=(connection, tipe, username_relay), daemon=True).start()
            print("=====")

```

Gambar 4. 13 *Manager Accept Connection*

Komponen *manager* memiliki kemampuan menerima koneksi *relay* atau *user*. *Relay* akan melalui konfigurasi terlebih dahulu sebelum dilayani sepenuhnya oleh *manager*. Konfigurasi berupa pemberian *username* untuk *relay* dan memberikan informasi mengenai *relay* lainnya kepada *relay* yang baru melakukan koneksi. Setelah itu *relay* akan dilayani dengan *threading* dengan fungsi *handle_component*. *User* akan langsung dilayani dengan *threading* dengan fungsi *handle_component*.

```

[root@jft manager]# python3 main.py
Local (y/n)?
Try to connecting.... 103.178.153.189 - 5000 (Tracker)
Connected
[{'ip_local': '103.166.156.127', 'port': 50250, 'type': 'manager', 'is_private': '0'}]
Manager listening...
=====
{'ip_local': '103.178.153.189', 'port': 38565, 'type': 'relay'}
KONFIGURASI RELAY
CONFIG RELAY: {'ip_local': '103.178.153.189', 'port': 38565, 'type': 'relay', 'username': '4*6f%0*r*', 'connection': 0}
Menyimpan komponen ['103.178.153.189', 38565, 'relay', '4*6f%0*r*', 0, 1726213162361, 1726213162361] ke dalam database!

Komponen yang diberikan kepada relay 4*6f%0*r* adalah:
[{'username': '4*6f%0*r*', 'ip_local': '103.178.153.189', 'port': 38565, 'type': 'relay', 'connection': 0}]
=====
{'error_msg': True, 'tipe': 'socket peer relay is closed'}
Komponen relay dengan username 4*6f%0*r* mengalami error....
Tipe yang terputus adalah relay
Putus karena socket peer relay is closed
Komponen yang masih berada dalam manager []

```

Gambar 4. 14 *Hasil Manager Accept Connection*

4.4.3 Pengelolaan Autentikasi

```
if(tipe == "client" and message.get("type") == "auth"):
    print("=====")
    print("AUTH")
    result = handle_auth(message, communicate, u_db, get_relay_with_less_connection_db)
    if(not result):
        username = None
    else:
        username = message.get("username")
        socket_user[username] = communicate
    print("=====")
    # continue
```

Gambar 4. 15 Manager Mengelola Autentikasi

User yang mengirimkan pesan dengan tipe *auth* akan dilayani untuk autentikasi.

```
if(register):
    # print(f'Terjadi permintaan registrasi dari user {username}\r\n')
    if(user):
        print(f'Terjadi error karena username {username} telah digunakan yang lain")
        # Pembuatan packet untuk pemberitahuan kepada user bahwa registrasi gagal
        objek = {"error": True, "msg": "Username used", "code": 409}
        print(f"Packet error yang dikirimkan kepada user {username}\r\n(objek)")
        send_message(communicate, objek)
        return False
    elif(not username or verify_whitespace(username)):
        print(f'Terjadi error karena username {username} kosong atau terdapat whitespace")
        # Pembuatan packet untuk pemberitahuan kepada user bahwa registrasi gagal
        objek = {"error": True, "msg": "Bad request", "code": 400}
        print(f"Packet error yang dikirimkan kepada user {username}\r\n(objek)")
        send_message(communicate, objek)
        return False
    password = message.get("password")
    if(not password or verify_whitespace(password)):
        print(f'Terjadi error karena password kosong atau terdapat whitespace")
        # Pembuatan packet untuk pemberitahuan kepada user bahwa registrasi gagal
        objek = {"error": True, "msg": "Bad request", "code": 400}
        print(f"Packet error yang dikirimkan kepada user {username}\r\n(objek)")
        send_message(communicate, objek)
        return False
    else:
        print(f'Terjadi permintaan registrasi dari user {username}\r\n')
        print(message)
        print("Registrasi berhasil\r\n")
        # Mendapatkan relay paling sedikit yang akan diberikan kepada user
        relay_for_user = get_relay_with_less_connection_db()
        # Pembuatan packet untuk pemberitahuan kepada user bahwa registrasi berhasil
        objek = {"error": False, "msg": "Account created", "code": 201, "component": relay_for_user}
        print(f"Packet success yang dikirimkan kepada user {username}\r\n(objek)")
        password = message.get("password")
        timestamp = get_timestamp()
        # Menyimpan user ke dalam database
        helper_registering_user(username, password, timestamp)
        send_message(communicate, objek)
        return True
```

Gambar 4. 16 Manager Logika Pengelolaan Registrasi User

Jika *user* memberikan pesan yang memiliki nilai *register* maka akan dilayani untuk proses registrasi. Ada beberapa kondisi yang harus disesuaikan oleh *user* ketika melakukan registrasi seperti *username* harus tidak digunakan oleh orang lain, *username* tidak boleh terdapat spasi atau kosong, *password* tidak boleh terdapat spasi atau kosong.

```

=====
{'type': 'auth', 'username': 'esa', 'password': '1234', 'register': True}
=====
AUTH
Terjadi error karena username esa telah digunakan yang lain
=====
{'error_msg': True, 'tipe': 'socket peer is closed'}
Komponen client dengan username "tanpa username" mengalami error....

```

Gambar 4. 17 Hasil *Manager* Mengelola Registrasi *Username* Terpakai

```

[root@jft manager]# python3 main.py
Local (y/n)?
Try to connecting... 103.178.153.189 - 5000 (Tracker)
Connected
[{'ip_local': '103.166.156.127', 'port': 57374, 'type': 'manager', 'is_private': '0'}]
Manager listening...
=====
{'ip_local': '103.178.153.189', 'port': 60727, 'type': 'relay'}
=====
{'ip_local': '192.168.1.6', 'port': 62998, 'type': 'client'}
=====
{'type': 'auth', 'username': None, 'password': None}
=====
AUTH
Terjadi permintaan login dari user None

Terjadi error karena username None kosong atau terdapat whitespace
=====

```

Gambar 4. 18 Hasil *Manager* Mengelola Data *User* Kosong atau Ada Spasi

```

{'type': 'auth', 'username': 'esa', 'password': '12345', 'register': True}
=====
AUTH
Terjadi permintaan registrasi dari user esa
{'type': 'auth', 'username': 'esa', 'password': '12345', 'register': True}
Registrasi berhasil
Relay koneksi paling sedikit [{'username': 'c$0508*{', 'ip_local': '103.178.153.189', 'port': 41940, 'type': 'relay', 'connection': 0}]
Menyimpan data user esa ke dalam database
[('esa', '12345', 'Feel happy using this application! ;D', 1, 1726214915071, 1726214915071)]

```

Gambar 4. 19 Hasil *Manager* Mengelola *User* Berhasil Registrasi

```

else:
    print(f"Terjadi permintaan login dari user {username}\r\n")
    if(not username or verify_whitespace(username)):
        print(f"Terjadi error karena username {username} kosong atau terdapat whitespace")
        # Pembuatan packet untuk pemberitahuan kepada user bahwa login gagal
        objek = {"error": True, "msg": "Bad request", "code": 400}
        print(f"Packet error yang dikirimkan kepada user {username}\r\n{objek}")
        send_message(communicate, objek)
        return False
    password = message.get("password")
    if(not password or verify_whitespace(password)):
        print(f"Terjadi error karena password kosong atau terdapat whitespace")
        # Pembuatan packet untuk pemberitahuan kepada user bahwa login gagal
        objek = {"error": True, "msg": "Bad request", "code": 400}
        print(f"Packet error yang dikirimkan kepada user {username}\r\n{objek}")
        send_message(communicate, objek)
        return False
    if(not user):
        print(f"Terjadi error karena username {username} yang dimasukkan tidak terdapat dalam sistem")
        # Pembuatan packet untuk pemberitahuan kepada user bahwa login gagal
        objek = {"error": True, "msg": "User not found", "code": 404}
        print(f"Packet error yang dikirimkan kepada user {username}\r\n{objek}")
        send_message(communicate, objek)
        return False
    password_db = user[1]

```

```

if(user[3]): # Cek status online
    if(not password == password_db):
        print(f"Terjadi error karena password tidak sesuai")
        # Pembuatan packet untuk pemberitahuan kepada user bahwa login gagal
        objek = {"error": True, "msg": "Bad request", "code": 400}
        print(f"Packet error yang dikirimkan kepada user {username}\r\n{objek}")
        send_message(communicate, objek)
        return False
    print(f"Terjadi error karena user mencoba login ke {username} yang sedang online")
    # Pembuatan packet untuk pemberitahuan kepada user bahwa login gagal
    objek = {"error": True, "msg": "Already logged in", "code": 409}
    print(f"Packet error yang dikirimkan kepada user {username}\r\n{objek}")
    send_message(communicate, objek)
    return False
if(password == password_db):
    print(f"{username} berhasil melakukan login\r\n")
    # Mendapatkan relay paling sedikit yang akan diberikan kepada user
    relay_for_user = get_relay_with_less_connection_db()
    # Pembuatan packet untuk pemberitahuan kepada user bahwa login berhasil
    objek = {"error": False, "msg": "Login successfull", "code": 200, "component": relay_for_user}
    # Memperbarui status dari user dari offline ke online
    print(f"Packet success yang dikirimkan kepada user {username}\r\n{objek}")
    update_status_online(["online", "updated_at"], (1, get_timestamp()), ['user_id'], (username,))
    send_message(communicate, objek)
    return True
else:
    print(f"Terjadi error karena password tidak sesuai")
    # Pembuatan packet untuk pemberitahuan kepada user bahwa login gagal
    objek = {"error": True, "msg": "Bad request", "code": 400}
    print(f"Packet error yang dikirimkan kepada user {username}\r\n{objek}")
    send_message(communicate, objek)
    return False

```

Gambar 4. 20 Manager Logika Pengelolaan *Login User*

Jika pesan yang dikirimkan *user* tidak memiliki nilai registrasi maka akan dilayani untuk proses *login*. Ada beberapa kondisi yang harus dipenuhi oleh *user* ketika ingin *login* yaitu *username* atau *password* tidak boleh ada spasi atau kosong, *username* dan *password* tidak boleh salah.

```

=====
{'type': 'auth', 'username': 'esa', 'password': '1234'}
=====
AUTH
Terjadi permintaan login dari user esa

Terjadi error karena password tidak sesuai
=====

```

Gambar 4. 21 Hasil *Manager Mengelola User Login Password Salah*

```

=====
{'type': 'auth', 'username': 'fajar', 'password': '12345'}
=====
AUTH
Terjadi permintaan login dari user fajar

Terjadi error karena username fajar yang dimasukkan tidak terdapat dalam sistem
=====

```

Gambar 4. 22 Hasil *Manager Mengelola User Login Username Salah*

```

=====
AUTH
Terjadi permintaan login dari user esa
esa berhasil melakukan login
Relay koneksi paling sedikit [{'username': 'GRj" g<s k"', 'ip_local': '103.178.153.189', 'port': 38754, 'type': 'relay', 'connection': 0}]
UPDATING STATUS ONLINE: [{'esa', '12345', 'Feel happy using this application! ;D', 1, 1726216487880, 1726216806453}]

```

Gambar 4. 23 Hasil Manager Mengelola User Login Berhasil

Ketika *user* berhasil melakukan registrasi atau *login*, maka *manager* akan secara otomatis memberikan *relay* dengan koneksi paling sedikit.

4.4.4 Pengelolaan Jumlah Koneksi Pada Relay

Komponen *manager* akan mendapatkan pesan NCIR (*new connection in relay*) atau ECIR (*end connection in relay*) dari *relay*, kemudian *manager* akan mengelola jumlah koneksi pada *relay* yang mengirimnya.

```

# new connection in relay
elif(message.get("message") and message.get("message").lower() == "ncir" and tipe == "relay"):
    print("=====")
    print("NCIR")
    print("Koneksi user terhubung dengan relay!")
    username_relay = message.get("username_relay")
    update_total_connection(username_relay, get_timestamp, 'NCIR')
    print("=====")

```

Gambar 4. 24 Manager Logika Pengelolaan NCIR

Ketika *manager* menerima pesan NCIR, maka *manager* akan menambahkan jumlah koneksi yang terkait dengan *relay* yang mengirimnya.

```

=====
{'message': 'ncir', 'username_relay': 'c$}0500`*{', 'tipe': 'relay'}
=====
NCIR
Koneksi user terhubung dengan relay!
NCIR_DB: [{'c$}0500`*{', '103.178.153.189', 41940, 'relay', 1, 1726214902120, 1726214916139}]

```

Gambar 4. 25 Hasil Manager Mengelola NCIR

```

# end connection in relay
elif(message.get("message") and message.get("message").lower() == "ecir" and tipe == "relay"):
    print(message)
    print("=====")
    print("ECIR")
    print("Koneksi user terputus dengan relay!")
    username_relay = message.get("username_relay")
    update_total_connection(username_relay, get_timestamp, 'ECIR')
    print("=====")

```

Gambar 4. 26 Manager Logika Pengelolaan ECIR

Ketika *manager* menerima pesan ECIR, maka *manager* akan mengurangi jumlah koneksi yang terkait dengan *relay* yang mengirimnya.

```

{'message': 'ecir', 'username_user': 'esa', 'username_relay': 'GRj" g<s k"' }
=====
ECIR
Koneksi user terputus dengan relay!
ECIR_DB: [{'GRj" g<s k"', '103.178.153.189', 38754, 'relay', 0, 1726216480440, 1726216494628}]
=====

```

Gambar 4. 27 Hasil Manager Mengelola ECIR

4.4.5 Pengelolaan Roster

```
# Mengelola yang berkaitan dengan roster
elif(message.get("stanza") and message.get("stanza").lower() == "iq" and message.get("namespace") and message.get("namespace").lower() == "roster"):
    print("=====")
    print("IQ, ROSTER")
```

Gambar 4. 28 Manager Mengelola Roster

Komponen *manager* ketika menerima pesan *stanza iq* dan *namespace roster* akan melayani pesan tersebut untuk pengelolaan *roster*.

```
# Mengelola user yang meminta rosternya
if(message.get("type") and message.get("type").lower() == "get"):
    print(f"Terjadi permintaan mendapatkan roster dari user {username}")
    objek = get_rosters(username, message)
    send_message(communicate, objek)
```

Gambar 4. 29 Manager Logika Pengelolaan Get Roster

Jika tipe yang dikirimkan adalah *get*, maka *manager* akan melayani pengambilan *roster* dari *user* yang melakukan *request*.

```
# Fungsionalitas untuk mendapatkan roster dari user
def get_rosters(username, msg):
    ...
    AMBIL YANG SUBSCRIPTIONNYA TO ATAU BOTH
    ...
    if (type(msg) != dict):
        return helper_error(msg, "modify", "bad-request")
    elif(msg.get("from") != username):
        return helper_error(msg, "auth", "forbidden")

    # Meminta data roster dari database, dengan tipe subscription 'to' atau 'both'
    roster = get_rosters_user(username, ('to', 'both'))

    msg = helper_parse_from_req_to_res(msg)
    msg["query"]["items"] = roster
    return msg
```

Gambar 4. 30 Manager Logika Get Roster Berdasarkan Subscription yang Sesuai

Manager akan mengambil *roster* yang memiliki tipe *subscription* adalah *to* atau *both* pada data *user* yang melakukan *request*. Tipe *to* adalah *user* telah menambahkan *user* lain ke dalam *roster*, dan *both* adalah *user* telah menambahkan *user* lain ke dalam *roster*-nya dan *user* lain telah menambahkan *user* ke dalam *roster*-nya.

```
=====
{'stanza': 'iq', 'namespace': 'roster', 'from': 'fajar', 'type': 'get', 'query': {'items': None}}
=====
IQ, ROSTER
Terjadi permintaan mendapatkan roster dari user fajar
RES_GR_DB: [{'jid': 'esa', 'name': 'ini esa', 'subscription': 'both'}]
=====
```

Gambar 4. 31 Hasil Manager Mengelola Get Roster

```

# Mengelola roster yang ingin menambahkan atau memperbarui roster item
elif(message.get("type") and message.get("type").lower() == "set" and not message.get("subscription")):
    print(f"Terjadi permintaan untuk menambahkan atau memperbarui roster item dari user {username}")
    jid_target = message.get('query').get('item').get('jid')
    objek = set_roster(username, message)
    if(objek['type'] != "error"):
        # Membuat presence bertipe "subscribed" untuk user pengirim
        packet_presence_subscribed = get_packet_subscribed_for_init_entity(username, jid_target)
        # Mengambil data roster item di dalam roster terbaru untuk user pengirim dari database
        item = get_roster_user(username, jid_target)
        print(f"ITEM {item}")
        print(f"GET ITEM DB ROSTER: {get_roster_user(username, jid_target)}")
        # Melengkapi packet presence bertipe "subscribed" dengan roster item terbaru
        packet_presence_subscribed["item"] = item
        # Mengirimkan packet lengkap presence bertipe "subscribed" kepada user pengirim
        send_message(communicate, packet_presence_subscribed)
        if(item.get("subscription") == "both" and get_user_online(jid_target)):
            packet_presence_subscribed_for_roster_item = get_packet_subscribed_for_init_entity(username, jid_target)
            item_for_roster_item = get_roster_user(jid_target, username)
            packet_presence_subscribed_for_roster_item["item"] = item_for_roster_item
            send_message(socket_user[jid_target], packet_presence_subscribed_for_roster_item)
        # Mengambil data presence terbaru dari roster item yang dikirimkan oleh user pengirim
        subscribed_entity_presence = get_presence_by_jid(jid_target)
        subscribed_entity_presence['to'] = username
        # Mengirimkan packet presence terbaru roster item kepada user pengirim
        send_message(communicate, subscribed_entity_presence)

```

Gambar 4. 32 Manager Logika Set atau Update Roster

Jika tipe yang dikirimkannya *set* maka *manager* akan mengelola pesan sebagai penambahan *roster* baru atau mengubah *roster* yang tersimpan. Ketika *user* berhasil menambahkan *user* lain ke dalam *roster*-nya, maka *manager* akan memberikan *presence* *user* tersebut kepada *user* yang melakukan *request*.

```

# Fungsionalitas untuk membantu menambahkan atau mengubah roster item
def set_to_rosters(username, item):
    jid_target = item["jid"]
    updating = False
    new = False
    # Mengecek apakah init_user sudah masuk ke roster_target
    tmp_item_jid = helper_check_and_get_roster(jid_target, username)
    # Logic komponen manager untuk menentukan apakah roster item akan ditambahkan atau diperbarui saja
    if(not tmp_item_jid):
        tmp_item_jid = {"jid": username, "subscription": None}
        new = True
    else:
        print("We will updating")
        updating = 1
    # Menentukan tipe subscription yang akan ditetapkan berdasarkan logic kepintaran pembeda
    if(not tmp_item_jid.get("subscription")):
        tmp_item_jid["subscription"] = "from"
    elif(tmp_item_jid.get("subscription") == "to"):
        updating = 2
        item["subscription"] = "both"
        tmp_item_jid["subscription"] = "both"
    if(updating == 2 or new):
        # Memanggil fungsionalitas untuk menyimpan atau mengubah roster item dari user's roster
        helper_set_rosters(username, item, updating)
        # Memanggil fungsionalitas untuk menyimpan atau mengubah roster item dari roster item's roster
        # dengan roster itemnya adalah user pengirim
        helper_set_rosters(jid_target, tmp_item_jid, updating)
    else:
        helper_set_rosters(username, item, updating)

```

Gambar 4. 33 Manager Logika Penentu Antara Set atau Update Roster

Fungsi diatas adalah pengelola pesan *roster* tipe *set*, pesan yang diterima akan dikelola sebagai permintaan menambahkan *roster* atau mengubah *roster*

yang tersimpan. Jika saat menambahkan *roster* ternyata *user* yang diinginkan telah menambahkan dirinya ke dalam *roster* mereka, maka status yang disimpan akan menjadi *both*. Jika saat menambahkan *roster* ternyata *user* yang diinginkan belum menambahkan dirinya ke dalam *roster* mereka, maka status yang disimpan akan menjadi *to*.

```
=====
{'stanza': 'iq', 'namespace': 'roster', 'from': 'fajar', 'type': 'set', 'query': {'item': {'jid': 'esa', 'name': 'author', 'subscription': 'to'}}}
=====
IQ, ROSTER
Terjadi permintaan untuk menambahkan atau memperbarui roster item dari user fajar
GRU DB: []
HST: fajar, {'jid': 'esa', 'name': 'author', 'subscription': 'to'}
PANJANG ROSTER: 6
COLUMN: ['id_owner_roster', 'jid', 'name', 'subscription', 'created_at', 'updated_at'], ['fajar', 'esa', 'author', 'to', 1726218794342, 1726218794342]
1
ROSTER IN DB: [(1, 'fajar', 'esa', 'author', 'to', 1726218794342, 1726218794342)]
HST: esa, {'jid': 'fajar', 'subscription': 'from'}
PANJANG ROSTER: 5
COLUMN: ['id_owner_roster', 'jid', 'subscription', 'created_at', 'updated_at'], ['esa', 'fajar', 'from', 1726218794345, 1726218794345]
2
ROSTER IN DB: [(1, 'fajar', 'esa', 'author', 'to', 1726218794342, 1726218794342), (2, 'esa', 'fajar', 'None', 'from', 1726218794345, 1726218794345)]
ITEM {'jid': 'esa', 'name': 'author', 'subscription': 'to'}
GET ITEM DB ROSTER: {'jid': 'esa', 'name': 'author', 'subscription': 'to'}
=====
```

Gambar 4. 34 Hasil Manager Pengelolaan Set Roster

```
=====
{'stanza': 'iq', 'namespace': 'roster', 'from': 'fajar', 'type': 'set', 'query': {'item': {'jid': 'esa', 'name': 'mengubah nama alias esa', 'subscription': 'to'}}}
=====
IQ, ROSTER
Terjadi permintaan untuk menambahkan atau memperbarui roster item dari user fajar
GRU DB: {'jid': 'fajar', 'name': 'None', 'subscription': 'from'}
We will updating
DATA IN UPDATING: fajar, {'jid': 'esa', 'name': 'mengubah nama alias esa', 'subscription': 'to'}
UPDATING DATA ROSTER: 1
[(1, 'fajar', 'esa', 'mengubah nama alias esa', 'to', 1726222786487, 1726222786487), (2, 'esa', 'fajar', 'None', 'from', 1726222786490, 1726222786490)]
ROSTER IN DB: [(1, 'fajar', 'esa', 'mengubah nama alias esa', 'to', 1726222786487, 1726222786487), (2, 'esa', 'fajar', 'None', 'from', 1726222786490, 1726222786490)]
ITEM {'jid': 'esa', 'name': 'mengubah nama alias esa', 'subscription': 'to'}
GET ITEM DB ROSTER: {'jid': 'esa', 'name': 'mengubah nama alias esa', 'subscription': 'to'}
=====
```

Gambar 4. 35 Hasil Manager Mengelola Update Roster

```
# Menghapus roster item dari roster user pengirim
elif(message.get("type") and message.get("type").lower() == "set" and message.get("subscription") and message.get("subscription").lower() == "remove"):
    print(f"Terjadi permintaan menghapus roster item dari user {username}")
    jid_target = message.get('query').get('item').get('jid')
    objek = delete_roster(username, message) # delete_roster_user
    if(objek['type'] != 'error'):
        # Membuat presence bertipe "unsubscribed" untuk user pengirim
        packet_presence_unsubscribed = get_packet_unsubscribed_for_init_entity(username, jid_target)
        # Mengirim packet presence kepada user pengirim
        send_message(communicate, packet_presence_unsubscribed)
    else:
        print(f"Terjadi error saat user {username} melakukan permintaan menghapus roster item")
        # Mengirimkan hasil dari menghapus roster item kepada user pengirim
        send_message(communicate, objek)
```

Gambar 4. 36 Manager Logika Menghapus Roster

Jika *user* menyertakan nilai *subscription* bernilai *remove*, maka *manager* akan mengelola pesan untuk melakukan penghapusan *roster* yang diinginkan, dan ketika berhasil melakukan penghapusan, maka *user* tidak akan mendapatkan *update presence* dari *user* yang dihapus.

```
=====
{'stanza': 'iq', 'namespace': 'roster', 'from': 'fajar', 'type': 'set', 'query': {'item': {'jid': 'esa', 'name': 'mengubah nama alias esa', 'subscription': 'to'}}}
=====
IQ, ROSTER
Terjadi permintaan menghapus roster item dari user fajar
GRU DB: {'jid': 'esa', 'name': 'mengubah nama alias esa', 'subscription': 'to'}
GRU DB: {'jid': 'fajar', 'name': 'None', 'subscription': 'from'}
1
ROSTER DB IN DELETE: [(2, 'esa', 'fajar', 'None', 'from', 1726222786490, 1726222786490)]
1
ROSTER DB IN DELETE: []
=====
```

Gambar 4. 37 Hasil Manager Mengelola Hapus Roster

4.4.6 Pengelolaan Presence

```
# Mengelola yang berkaitan dengan presence
elif(message.get("stanza") and message.get("stanza").lower() == "presence"):
```

Gambar 4. 38 Manager Pengelolaan Presence

Jika user mengirim pesan *stanza presence*, maka *manager* akan mengelola pesan tersebut sebagai proses *presence*.

```
# Mengelola init presence
if(not message.get('type') and not message.get("bio") and not message.get("to")):
    print(f"Terjadi permintaan init presence dari user dengan username {username}")
    timestamp = get_timestamp()
    init_presence(socket_user, username, timestamp)
```

Gambar 4. 39 Manager Logika Pengelolaan Init Presence

Jika pesan tersebut hanya berisikan nilai *stanza presence*, maka *manager* akan mengelolanya sebagai *initial presence*. *Initial presence* adalah pesan yang dikirimkan oleh user ketika baru saja *register* atau *login*, fungsinya adalah memberikan status *online* kepada user lain yang telah menambahkannya ke dalam *roster*.

```
=====
{'stanza': 'presence'}
=====
PRESENCE
Terjadi permintaan init presence dari user dengan username esa
UPDATING STATUS ONLINE: [('esa', '12345', 'Feel happy using this application! ;D', 1, 1726214915071, 1726214916245)]
RES GR_DB: []
RES GR_DB: []
=====
```

Gambar 4. 40 Hasil Manager Mengelola Init Presence

```
# Mengelola ketika user memperbarui bio-nya
elif(message.get('bio')):
    print(f"Terjadi permintaan perubahan bio dari user dengan username {username}")
    timestamp = get_timestamp()
    set_my_bio(socket_user, username, message, timestamp)
```

Gambar 4. 41 Manager Pengelolaan Update Bio User

Jika pesan yang dikirimkan oleh user memiliki nilai *bio*, maka *manager* akan mengelola permintaan sebagai pembaruan status user.

```
# Fungsionalitas untuk mengelola permintaan user mengubah bio
def set_my_bio(socket_user, username, msg, timestamp):
    if (type(msg) != dict):
        return helper_error(msg, "modify", "bad-request")
    elif(username != msg.get("from")):
        return helper_error(msg, "auth", "forbidden")
    update_bio(['bio', 'updated_at'], (msg.get('bio'), timestamp,), ['user_id'], (username,))
    my_presence = get_presence_by_jid(username)
    roster_db_from = get_rosters_user(username, ("from", "both",))
    if(not roster_db_from):
        return
    send_to_roster_subscription_from(roster_db_from, socket_user, my_presence, False)
```

Gambar 4. 42 Manager Logika Update Bio User

Manager akan memperbarui status dari user, kemudian mengirimkan status terbaru itu kepada user lain yang telah menambahkannya ke dalam roster.

```
=====
{'stanza': 'presence', 'from': 'esa', 'bio': 'lagi makan kebab'}
=====
PRESENCE
Terjadi permintaan perubahan bio dari user dengan username esa
UPDATING BIO: [['esa', '12345', 'lagi makan kebab', 1, 1726222767000, 1726223223165], ('fajar', '12345', 'Feel happy using this application! ;D', 1, 1726222779046, 1726222779175)]
RES GR_DB: [{'jid': 'fajar', 'name': 'ini fajar', 'subscription': 'both'}]
ROSTER FROM AND BOTH IN DB: [{'jid': 'fajar', 'name': 'ini fajar', 'subscription': 'both'}]
=====
```

Gambar 4. 43 Hasil Manager Pengelolaan Update Bio User

```
# Mengelola ketika user membuat permintaan directed presence
elif(message.get('to')):
    jid_target = message.get('to')
    print(f"Terjadi permintaan directed presence yang dilakukan oleh user {username} dengan target user adalah {jid_target}")
    if(not get_user_by_username(jid_target)):
        print(f"Terjadi error saat permintaan directed presence karena target user {jid_target} tidak terdaftar di dalam sistem")
        objek_presence_error = {'stanza': 'presence', 'type': 'error', 'to': username, 'target': jid_target}
        send_message(communicate, objek_presence_error)
        continue
    # Kirim status presence init_entity ke target_entity jika target_entity online
    if(get_user_online(jid_target)):
        print(f"Mengirimkan directed presence kepada target user {jid_target} karena target user sedang online")
        init_entity_presence = get_presence_by_jid(username)
        init_entity_presence['directed_entity'] = True
        send_presence_to_someone(jid_target, socket_user, init_entity_presence)
    # Ambil status dari target_entity dan kirim ke init_entity
    target_entity_presence = get_presence_by_jid(jid_target)
    target_entity_presence['directed_entity'] = True
    print(target_entity_presence)
    send_presence_to_someone(username, socket_user, target_entity_presence)
```

Gambar 4. 44 Manager Logika Directed Presence

Jika pesan yang dikirimkan user memiliki nilai *to*, maka *manager* akan mengelola pesan sebagai *directed presence*. *Directed presence* adalah proses meminta *presence* user lain tanpa menambahkan user yang diinginkan ke dalam roster, proses ini hanya berjalan satu kali dan tidak berkelanjutan. Ketika meminta *presence* user lain, *manager* akan memberikan *presence* user tersebut kepada user yang diinginkan jika sedang *online*.

```
=====
{'stanza': 'presence', 'to': 'esa'}
=====
PRESENCE
Terjadi permintaan directed presence yang dilakukan oleh user fajar dengan target user adalah esa
Mengirimkan directed presence kepada target user esa karena target user sedang online
{'stanza': 'presence', 'from': 'esa', 'username': 'esa', 'bio': 'mengubah bio saat fajar udah unsubscribe', 'online': 1, 'updated_at': 1726223542222, 'directed_entity': True}
=====
```

Gambar 4. 45 Hasil Manager Mengelola Directed Presence

```
# Mengelola ketika user membuat permintaan presence tipe "unavailable" atau logout
elif(message.get('type') == 'unavailable'):
    print(f"Terjadi permintaan presence bertipe 'unavailable' dari user {username}")
    lost_connection("Memberikan presence tipe unavailable", tipe, username_relay, username)
    error = True
```

Gambar 4. 46 *Manager Pengelolaan Logout User*

Jika pesan yang dikirimkan *user* memiliki tipe bernilai *unavailable*, maka *manager* akan mengelola pesan ini sebagai permintaan *logout user*.

```
def lost_connection(reason, tipe, username_relay, username):
    print(f"Tipe yang terputus adalah {tipe}")
    print(f"Putus karena {reason}")
    if(tipe and tipe.lower() == "relay"):
        # Menghapus komponen relay dari database
        delete_components_db_by_id(c_db, tuple_condition=username_relay)
    if(username):
        # Menghapus koneksi dari user
        socket_user[username].close()
        del socket_user[username]
        # Konfigurasi user yang logout atau mengalami error
        timestamp = get_timestamp()
        logout(socket_user, username, timestamp)
```

Gambar 4. 47 *Manager Logika Pengelolaan Logout User*

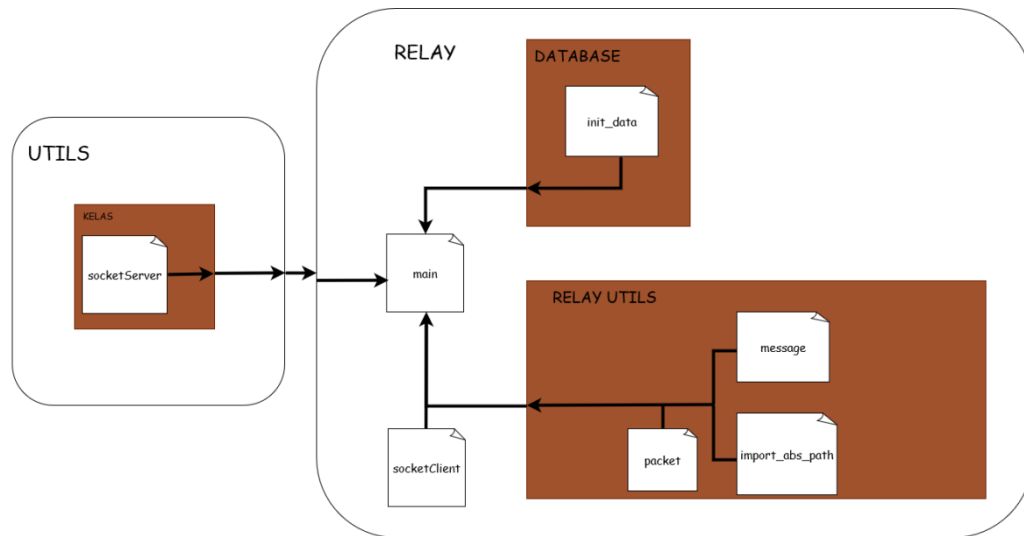
Manager akan menghapus dan memutus koneksi dengan *user* yang melakukan *logout* dan mengubah status *user* dari *online* menjadi *offline*. Kemudian *manager* akan mengirimkan status terbaru dari *user* kepada *user* lain yang telah menambahkannya ke dalam *roster* mereka.

```
=====
{'stanza': 'presence', 'from': 'esa', 'type': 'unavailable'}
=====
PRESENCE
Terjadi permintaan presence bertipe 'unavailable' dari user esa
UPDATING STATUS ONLINE: [(C'esa', '12345', 'Lagi makan kebab', 0, 1726222767000, 1726223301560), (C'fajar', '12345', 'Feel happy using this application! ;D',
1, 1726222779046, 1726222779175)]
RES GR_DB: [{'jid': 'fajar', 'name': 'ini fajar', 'subscription': 'both'}]
ROSTER FROM AND BOTH IN DB: [{'jid': 'fajar', 'name': 'ini fajar', 'subscription': 'both'}]
=====
```

Gambar 4. 48 *Hasil Manager Mengelola Logout User*

4.5 Implementasi Komponen *Relay*

4.5.1 Diagram Komponen *Relay*



Gambar 4. 49 Diagram Komponen *Relay*

Komponen *relay* memiliki *file* utama yaitu *main.py* yang terdapat pada *folder relay*. Memiliki *file socketClient* yang berfungsi untuk memberikan *relay* kemampuan menjadi *client* komponen lain. Memiliki *file* untuk mengelola pesan yang masuk melalui jaringan, mengelola *stanza message*, *import file* yang bukan dari lingkungannya, yang terdapat pada *folder relay/relay_utils*.

4.5.2 Inisiasi *Relay*

```
s, message = connect_to_tracker()
relay_to_tracker = s.socket
my_ip = s.localAddress
```

Gambar 4. 50 *Relay* Konek ke *Tracker*

Relay akan melakukan koneksi kepada komponen *tracker*.

```
[root@jft relay]# python3 main.py
Local (y/n)?
Try to connecting.... 103.178.153.189 - 5000 (Tracker)
Connected
Komponen dari tracker:
[{'ip_local': '103.166.156.127', 'port': 59441, 'type': 'manager', 'is_private': '0'}, {'ip_local': '103.178.153.189', 'port': 51541, 'type': 'relay', 'is_private': '0'}, {'ip_local': '103.166.156.127', 'port': 34462, 'type': 'relay', 'is_private': '0'}]
```

Gambar 4. 51 Hasil *Relay* Konek ke *Tracker*

```

# Connect ke manager
while not s2:
    target = None
    for m in message:
        if(m.get("type").lower() == "manager"):
            target = m
            break
    if(target):
        ip = target.get("ip_local")
        port = target.get("port")
        s2 = SocketClientRelay(ip, port, tipe="Manager")
        relay_to_manager = s2.socket
        my_ip = s2.localAddress
        ip_is_private = ipaddress.ip_address(my_ip[0]).is_private
        objek = {
            "ip_local": my_ip[0],
            "port": my_ip[1],
            "type": "relay"
        }
        send_message(relay_to_manager, objek)
        messages = get_message_manager(relay_to_manager)
        for msg in messages:
            msg_from_manager = json.loads(msg)
            print(f"Pesan dari manager: {msg_from_manager}")
            print("=====")
            # Melakukan konfigurasi awal saat relay pertama kali online, dengan daftar komponen yang diberikan oleh manager
            config_starter_relay(msg_from_manager)
            my_port = my_ip[1]
            # r = SocketServerRelay(my_port)
            r = SocketServer(my_port)
            relay = r.socket
        if(not s2 or not len(message)):
            print("Belum ada manager!")
            time.sleep(10)
            # Meminta manager kepada tracker
            message = get_manager(relay_to_tracker)

```

Gambar 4. 52 Relay Logika Konek ke Manager, Konfigurasi dengan Relay Lainnya, dan Menjadi Server

Setelah terkoneksi dengan *tracker*, *relay* akan menunggu balasan dari *tracker* untuk alamat *manager*. Jika belum terdapat komponen *manager*, maka *relay* akan meminta secara berkala kepada *tracker* untuk alamat *manager*. Saat sudah mendapatkan alamat *manager*, kemudian *relay* akan melakukan koneksi kepada *manager*.

```

=====
Try to connecting.... 103.166.156.127 - 53719 (Manager)
Connected
Pesan dari manager: {'components': [{'username': '&(>?e_QvB1', 'ip_local': '103.178.153.189', 'port': 51476, 'type': 'relay', 'connection': 1}, {'username': 'zwNs?R?-gw', 'ip_local': '103.166.156.127', 'port': 52662, 'type': 'relay', 'connection': 0}], 'username': 'zwNs?R?-gw'}
=====

```

Gambar 4. 53 Hasil Relay Konek ke Manager

Setelah berhasil terkoneksi dengan *manager*, maka *relay* akan melakukan konfigurasi awal, yaitu mengoneksikan dirinya dengan *relay-relay* lainnya, jika terdapat *relay* lain yang sudah terkoneksi dengan *manager* sebelumnya.

```

=====
Melakukan koneksi dengan relay lainnya
Try to connecting.... 103.178.153.189 - 51476 (Relay)
Connected
Pesan dari relay senior: {'relay_username': '&(>?e_QvB1', 'username_users': ['esa']}
=====

```

Gambar 4. 54 Hasil Relay Konek ke Relay Lainnya

Jika *relay* yang lain memiliki *user* yang terkoneksi ke dirinya, maka *relay* akan menyimpan informasi tersebut ke dalam datanya. Setelah berhasil melakukan semuanya, *relay* akan menjadi server dan dapat menerima koneksi dari *relay* lainnya atau *user*.

```
=====
Connection Relay Konfigurasi: {'&(>?e_QvB1': <socket.socket fd=5, family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0, laddr=('103.166.156.127', 50712), raddr=('103.178.153.189', 51476)>}
User in Another Relay Konfigurasi: {'esa': '&(>?e_QvB1'}
=====
```

Gambar 4. 55 Hasil Relay Menyimpan Data User yang Terkoneksi Pada Relay Lainnya

```
# Fungsionalitas untuk menerima segala koneksi TCP yang ingin terkoneksi dengan dirinya
while True:
    connection, address = relay.accept()
    messages = get_message(connection)
    for msg in messages:
        # print(msg)
        message = json.loads(msg)
        if message.get("username"):
            print(f"Mendapatkan koneksi baru dari user {msg}")
            username = message.get("username")
            # Menyimpan user ke dalam daftar koneksinya
            connections[username] = connection
            # Membuat layanan eksklusif untuk mengelola komponen user
            threading.Thread(target=handle_component_user, args=(connection, my_username, username), daemon=True).start()
            objek = {
                "message": "ncir", # new connection in relay
                "username_relay": my_username,
                "tipe": "relay"
            }
            send_message(relay_to_manager, objek)
            send_new_user_to_another_relay(username, my_username)
        elif message.get("type") == "relay":
            print(f"Mendapatkan koneksi baru dari relay junior {msg}")
            uname_relay = message.get("relay_username")
            # Memanggil fungsi untuk menyimpan relay yang baru terkoneksi
            config_new_relay(uname_relay, connection)
            usernames = list(connections.keys())
            # Membuat objek yang akan dikirimkan kepada relay yang baru terkoneksi
            objek = {
                "relay_username": my_username,
                "username_users": usernames
            }
            send_message(connection, objek)
            # Membuat layanan eksklusif untuk mengelola komponen relay
            threading.Thread(target=handle_component_relay, args=(connection, my_username, ), daemon=True).start()
            print('=====')
        elif message.get("msg") == "Hello relay!":
            objek = {"msg": "Hallo target"}
            send_message(connection, objek)
```

Gambar 4. 56 Relay Accept Connection

Jika *relay* menerima koneksi dengan pesan bertipe *relay*, maka *relay* akan menyimpan *username* dan jalur *socket relay* tersebut, kemudian memberikan seluruh *user* yang terkoneksi dengan dirinya kepada *relay* tersebut. Setelah itu *relay* akan menangani *relay* tersebut dengan *threading* yang memiliki fungsi *handle_component_relay*.

```
Mendapatkan koneksi baru dari relay junior {'ip_local': '103.166.156.127', 'port': 41777, 'type': 'relay', 'is_private': false, 'relay_username': 'zwNs7R?~g
w'})
Koneksi dengan relay lainnya saat ini:
{'zwNs7R?~gw': <socket.socket fd=7, family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0, laddr=('103.178.153.189', 51476), raddr=('103.166.156.127', 50712)>}
=====
```

Gambar 4. 57 Hasil Relay Accept Connection Relay Lainnya

Jika *relay* menerima koneksi dengan pesan *username*, maka *relay* akan menyimpan koneksi *user* tersebut dan menanganinya dengan *threading* fungsi *handle_component_user*, kemudian memberikan pesan NCIR kepada *manager* dan pesan *new user* kepada *relay* lain yang terkoneksi dengannya.

```
=====
Mendapatkan koneksi baru dari user {"username": "fajar", "message": "hello!"}
```

Gambar 4. 58 Hasil *Relay Accept Connection User*

4.5.3 Pengelolaan User Antar Relay

```
# Mengelola packet dengan nilai message "new user"
elif(message.get("message") and message.get("message").lower() == "new user"):
    print('=====')
    print("Relay mendapatkan pesan 'new user' dari relay lainnya")
    print(message)
    relay_uname = message.get("relay_username")
    username_user = message.get("username_user")
    print(f"Koneksi {username_user} kepada {relay_uname} telah tersambung!")
    # Mendaftarkan user ke dalam daftar koneksi relay pengirim
    user_in_another_relay[username_user] = relay_uname
    print(f"User didalam relay {relay_uname} adalah {user_in_another_relay}")
    print('=====')
```

Gambar 4. 59 *Relay Logika Pengelolaan User Baru dari Relay Lainnya*

Jika *relay* menerima pesan *new user* dari *relay* lainnya, maka *relay* akan mengelola dengan menyimpan *username* dari *user* yang baru saja terkoneksi dengan *relay* lain tersebut.

```
=====
Relay mendapatkan pesan 'new user' dari relay lainnya
{'message': 'new user', 'username_user': 'fajar', 'relay_username': 'zwNs?R?~gw'}
Koneksi fajar kepada zwNs?R?~gw telah tersambung!
User didalam relay zwNs?R?~gw adalah {'fajar': 'zwNs?R?~gw'}
=====
```

Gambar 4. 60 Hasil *Relay Mengelola User Baru dari Relay Lainnya*

```
# Mengelola packet dengan nilai message "end user"
elif(message.get("message") and message.get("message").lower() == "end user"):
    print('=====')
    print("Relay mendapatkan pesan 'end user' dari relay lainnya")
    print(message)
    relay_uname = message.get("username_relay")
    username_user = message.get("username_user")
    print(f"Koneksi {username_user} kepada {relay_uname} telah terputus!")
    # Menghapus user dari daftar koneksi relay pengirim
    del user_in_another_relay[username_user]
    print('=====')
```

Gambar 4. 61 *Relay Logika Pengelolaan User Terputus dari Relay Lainnya*

Jika *relay* menerima pesan *end user* dari *relay* lainnya, maka *relay* akan mengelola dengan menghapus *username* dari *user* yang baru saja terputus dengan *relay* lain tersebut.

```
=====
Relay mendapatkan pesan 'end user' dari relay lainnya
{'message': 'end user', 'username_user': 'esa', 'username_relay': '&(>?e_QvB1'}
Koneksi esa kepada &(>?e_QvB1 telah terputus!
=====
```

Gambar 4. 62 Hasil *Relay* Mengelola *User* Terputus dari *Relay* Lainnya

```
print(e)
print("Connection end...")
# Objek yang akan diberitahukan kepada relay lainnya
objek = {
    "message": "end user",
    "username_user": user_username,
    "username_relay": relay_username
}
# Menghapus user dari daftar user yang terkoneksi
del connections[user_username]
# Memberi tahu relay lain bahwa ada user yang terputus koneksinya dengan dirinya
for r in connection_relay.values():
    send_message(r, objek)
# Objek messagenya diganti "ecir" dan dikirim kepada manager
objek["message"] = "ecir"
# objek[tipe]
send_message(relay_to_manager, objek)
break
```

Gambar 4. 63 *Relay* Logika Mengirimkan *User* Terputus

Relay yang koneksi *user*-nya terputus akan mengirimkan pesan *end user* kepada semua *relay* yang terkoneksi dengan dirinya, menghapus *username* *user* dari daftar terkoneksiya sendiri, dan mengirimkan pesan ECIR kepada *manager*.

4.5.4 Pengelolaan Stanza Message User

```

elif(message.get("stanza") and message.get("stanza") == "message"):
    print('=====')
    print(f'Menerima packet stanza message dari user')
    print(message)
    # Saving message to database
    initiator = message.get("from")
    target = message.get("to")
    message_s = message.get("body")
    time_send = message.get("time_send")
    save_message_to_db([initiator, target, message_s, time_send])
    # End saving message to database
    connection_target = connections.get(target)
    # Target dalam relay yang sama
    if(connection_target):
        print("Mengirim pesan kepada target pada relay yang sama")
        send_message_to_target(connection_target, message)
    # Target dalam relay yang berbeda
    else:
        relay_user = user_in_another_relay.get(target)
        print(f"Meneruskan pesan kepada relay {relay_user}")
        if(not relay_user):
            print("Terjadi error karena user yang dituju tidak terdapat di dalam sistem")
            initiate_user = message.get("from")
            # Mengirim packet error kepada user
            send_packet_error(communicate, 404, initiate_user)
            print('=====')
            continue
        socket_relay = connection_relay.get(relay_user)
        send_message_to_target(socket_relay, message)
    print('=====')

```

Gambar 4. 64 Relay Logika Pengelolaan Stanza Message dari User

Jika *relay* mendapatkan *stanza message* dari *user*, maka *relay* akan mengelolanya dengan cara menyimpan pesan yang diterima, kemudian mencari apakah *user* yang diinginkan berada pada *relay* yang sama atau tidak, atau tidak ada sama sekali di dalam sistem.

```

=====
Menerima packet stanza message dari user
{'stanza': 'message', 'from': 'esa', 'to': 'sukma', 'lang': 'id', 'body': 'halo sukma'}
Meneruskan pesan kepada relay None
Terjadi error karena user yang dituju tidak terdapat di dalam sistem
Packet error untuk user esa
{'stanza': 'message', 'from': 'relay', 'to': 'esa', 'error': True, 'body': 'User not found or User not online'}
=====

```

Gambar 4. 65 Hasil Relay Mengelola Stanza Message User Tujuan Tidak Terdaftar

```

=====
Menerima packet stanza message dari user
{'stanza': 'message', 'from': 'esa', 'to': 'sukma', 'lang': 'id', 'body': 'halo sukma'}
Mengirim pesan kepada target pada relay yang sama
=====

```

Gambar 4. 66 Hasil Relay Mengelola Stanza Message User Berada Pada Relay yang Sama

```

=====
Menerima packet stanza message dari user
{'stanza': 'message', 'from': 'esa', 'to': 'fajar', 'lang': 'id', 'body': 'halo fajar'}
Meneruskan pesan kepada relay zwNs?R?~gw
=====
Relay mendapatkan packet nilai success 1 dan nilai activity 'messsage from another relay'
{'from': 'esa', 'to': 'fajar', 'activity': 'message from another relay', 'success': 1}
=====

```

Gambar 4. 67 Hasil *Relay* Mengelola *Stanza Message* Untuk *Relay* Lainnya

4.5.5 Pengelolaan *Stanza Message* Antar *Relay*

```

# Mengelola packet yang masuk adalah stanza message
elif(message.get("stanza") and message.get("stanza") == "message"):
    print('=====')
    print(f"Relay mendapatkan stanza message dari relay {connected_relay_username}")
    print(message)
    target = message.get("to")
    connection_target = connections.get(target)
    # Memanggil fungsi untuk mengirim stanza message kepada target
    result = send_message_to_target(connection_target, message)
    # Membuat objek untuk mengembalikan hasil dari pengiriman pesan, dengan nilai activity adalah "message from another relay"
    objek = {"from": message.get("from"), "to": message.get("to"), "activity": "message from another relay"}
    # Validitas apakah error atau success
    if(not result):
        objek["error"] = 1
    else:
        objek["success"] = 1
    send_message(communicate, objek)
    print('=====')

```

Gambar 4. 68 *Relay* Logika Pengelolaan *Stanza Message* Dari *Relay* Lainnya

Jika *relay* menerima *stanza message* dari *relay* lainnya, maka *relay* akan meneruskannya pada *user* yang dituju.

```

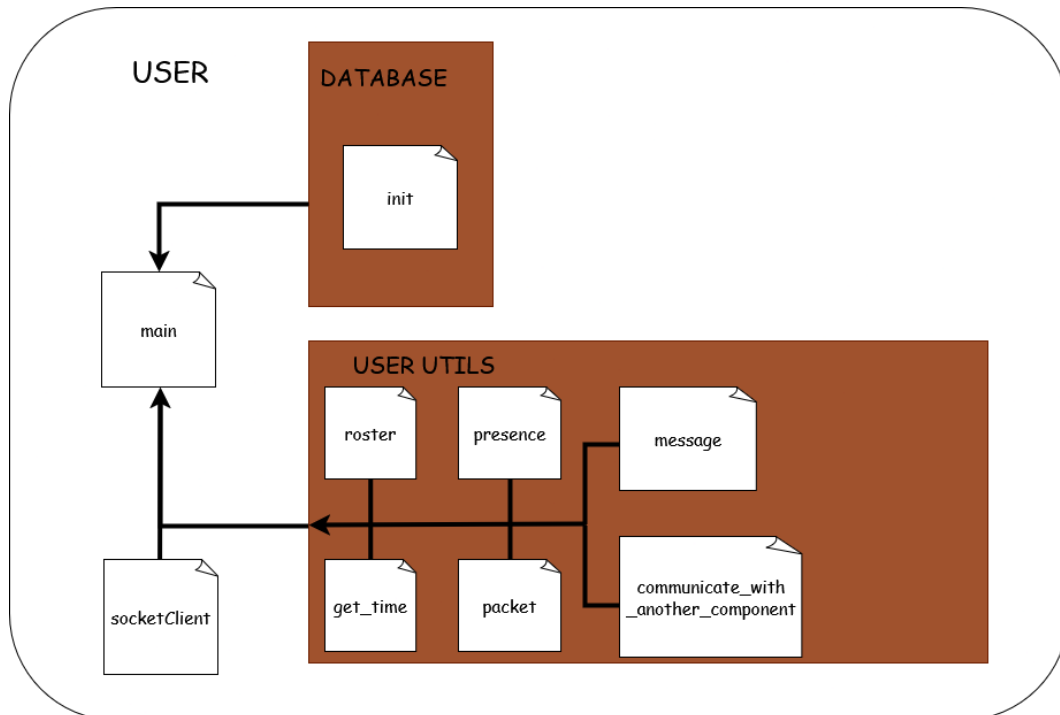
=====
Relay mendapatkan stanza message dari relay &(>?e_QvB1
{'stanza': 'message', 'from': 'esa', 'to': 'fajar', 'lang': 'id', 'body': 'halo fajar'}
=====

```

Gambar 4. 69 Hasil *Relay* Meneruskan *Stanza Message* Kepada *User* Tujuan

Jika terjadi *error* maka *relay* akan memberikan pesan *error* kepada *relay* pengirim.

4.6 Implementasi Komponen Penguji atau User



Gambar 4. 70 Diagram Komponen User

Implementasi komponen *user* ditujukan untuk menguji setiap fungsionalitas yang terdapat dalam daftar setiap komponen yang berhubungan dengan *user*. Metode atau fungsionalitas yang terdapat di dalam *user* untuk memenuhi perannya adalah sebagai berikut.

1. Melakukan koneksi *socket* TCP kepada komponen *tracker*. Mengirimkan *packet* kepada TCP dengan nilai kunci *type* adalah *client*.
2. Melakukan koneksi *socket* TCP kepada komponen *manager*. Mengirimkan *packet* kepada TCP dengan nilai kunci *type* adalah *client*.
3. Mampu mengirimkan *packet* untuk registrasi kepada komponen *manager*.
4. Mampu mengirimkan *packet* untuk *login* kepada komponen *manager*.
5. Melakukan koneksi *socket* TCP kepada komponen *relay*. Mengirimkan *packet* kepada TCP dengan nilai kunci *username* adalah *username* dari *user*.
6. Melakukan konfigurasi awal dengan mengirimkan permintaan *roster*-nya dan inisialisasi *presence* kepada komponen *manager*.
7. Melakukan permintaan *roster* kepada komponen *manager*.

8. Melakukan permintaan menambahkan *roster item* ke dalam *roster*-nya kepada komponen *manager*.
9. Melakukan permintaan mengubah *name* dari *roster item* kepada komponen *manager*.
10. Melakukan permintaan menghapus *roster item* dari *roster*-nya kepada komponen *manager*.
11. Melakukan permintaan *directed presence* terhadap suatu *username* tertentu kepada komponen *manager*.
12. Melakukan permintaan memperbarui bio dirinya kepada komponen *manager*.
13. Melakukan permintaan *logout* atau *presence* bertipe “*unavailable*” kepada komponen *manager*.
14. Melakukan pengiriman pesan kepada *username* tertentu kepada komponen *relay*.
15. Menerima segala *packet* pesan yang masuk dari komponen *tracker*, *manager*, dan *relay*.

Adapun cara atau standar pesan pada *user* untuk melakukan komunikasi dengan sistem adalah sebagai berikut.

1. Menambahkan *Roster*

```
{
  "stanza": "iq",
  "namespace": "roster",
  "from": "user_initiator",
  "type": "set",
  "query": {
    "item": {
      "jid": "user_target",
      "name": "alias_name_for_user_target",
      "subscription": "to"
    }
  }
}
```

Gambar 4. 71 Standar Pesan Menambahkan *Roster*

Standar pesan ini harus dikirimkan kepada komponen *manager* sebagai *payload*. Mengirimkan pesan ini kepada *manager*, menandakan bahwa *user* ingin menambahkan *user* lainnya ke dalam *roster* yang dimilikinya.

2. Meminta Daftar *Roster*

```
{
  "stanza": "iq",
  "namespace": "roster",
  "from": "user_initiator",
  "type": "get",
  "query": {"items": null}
}
```

Gambar 4. 72 Standar Pesan Meminta Daftar *Roster*

Standar pesan ini harus dikirimkan kepada komponen *manager* sebagai *payload*. Mengirimkan pesan ini kepada *manager*, menandakan bahwa *user* ingin meminta daftar *roster* yang dimilikinya dalam sistem.

3. Memperbarui Alias Roster

```
{
  "stanza": "iq",
  "namespace": "roster",
  "from": "user_initiator",
  "type": "set",
  "query": {
    "item": {
      "jid": "user_target",
      "name": "new_name_user_target",
      "subscription": "to"
    }
  }
}
```

Gambar 4. 73 Standar Pesan Memperbarui Alias Roster

Standar pesan ini harus dikirimkan kepada komponen *manager* sebagai *payload*. Mengirimkan pesan ini kepada *manager*, menandakan bahwa *user* ingin memperbarui nama panggilan dari *user* tertentu dalam *roster*-nya.

4. Menghapus *Roster*

```
{
  "stanza": "iq",
  "namespace": "roster",
  "from": "user_initiator",
  "type": "set",
  "query": {
    "item": {
      "jid": "user_target",
      "name": "name_user_target",
      "subscription": "to"
    }
  },
  "subscription": "remove"
}
```

Gambar 4. 74 Standar Pesan Menghapus *Roster*

Standar pesan ini harus dikirimkan kepada komponen *manager* sebagai *payload*. Mengirimkan pesan ini kepada *manager*, menandakan bahwa *user* ingin menghapus *user* lainnya dari *roster* yang dimilikinya.

5. Inisialisasi *Presence*

```
{
  "stanza": "presence"
}
```

Gambar 4. 75 Standar Pesan Inisialisasi *Presence*

Standar pesan ini harus dikirimkan kepada komponen *manager* sebagai *payload*. Mengirimkan pesan ini kepada *manager*, menandakan bahwa *user* baru saja melakukan *login*, dan ingin memberitahu kepada sistem bahwa dirinya sudah *online*.

6. *Directed Presence*

```
{
  "stanza": "presence",
  "to": "user_target"
}
```

Gambar 4. 76 Standar Pesan *Directed Presence*

Standar pesan ini harus dikirimkan kepada komponen *manager* sebagai *payload*. Mengirimkan pesan ini kepada *manager*, menandakan

bahwa *user* ingin meminta status dari *user* lainnya, yang belum *user* tambahkan ke *roster*-nya.

7. Memperbarui Bio

```
{
  "stanza": "presence",
  "bio": "value_of_bio"
}
```

Gambar 4. 77 Standar Pesan Memperbarui Bio

Standar pesan ini harus dikirimkan kepada komponen *manager* sebagai *payload*. Mengirimkan pesan ini kepada *manager*, menandakan bahwa *user* ingin mengubah bio yang dimilikinya di dalam sistem.

8. Logout dari Sistem

```
{
  "stanza": "presence",
  "type": "unavailable"
}
```

Gambar 4. 78 Standar Pesan Melakukan Logout

Standar pesan ini harus dikirimkan kepada komponen *manager* sebagai *payload*. Mengirimkan pesan ini kepada *manager*, menandakan bahwa *user* ingin melakukan *logout* dari sistem.

9. Mengirimkan Pesan Kepada User Lain

```
{
  "stanza": "message",
  "from": "user_initiator",
  "to": "user_target",
  "lang": "id",
  "body": "value_of_message",
  "time_send": "timestamp_now"
}
```

Gambar 4. 79 Standar Pesan Mengirimkan Pesan Kepada User Lain

Standar pesan ini harus dikirimkan kepada komponen *relay* sebagai *payload*. Mengirimkan pesan ini kepada *relay*, menandakan bahwa *user* ingin melakukan komunikasi dengan *target* yang ditulis dalam nilai “to”.

4.7 Pengujian Proses Komunikasi

Proses komunikasi antar komponen dimulai dari kemampuan setiap komponen membentuk jaringan komunikasi tipe *long-lived TCP connection*. Berdasarkan pengujian untuk komponen *tracker*, komponen ini mampu memberikan kebutuhan komponen *manager*, *relay*, dan *user* dengan sesuai. Komponen *manager* juga mampu menerima dan memberikan pesan balasan yang sesuai kepada komponen *relay* dan *user*.

Meskipun komponen *relay* berada pada VPS yang berbeda, komponen *relay* mampu menangani komunikasi antar *relay* maupun dengan *user* secara baik dan benar. Komunikasi yang dilakukan *user* untuk menguji kemampuan komunikasi komponen *manager* dan *relay* saat diberikan sebuah pesan kepada mereka, memberikan hasil balasan yang sesuai dengan permintaan *user* pengirim.

4.8 Pengujian Konsistensi Data

Arsitektur *microservice* yang digunakan untuk memecah setiap komponen tidak menyebabkan data hilang ketika dilakukan pengiriman. Hal ini dibuktikan dengan setiap permintaan yang dilakukan selalu menghasilkan hasil yang sesuai. Baik itu komunikasi antara komponen *tracker* dengan lainnya, komponen *manager* dengan lainnya, ataupun komponen *relay* dengan lainnya. Tidak ada data yang hilang ketika melakukan pengiriman data.

4.9 Pengujian Fungsionalitas

Fungsionalitas yang berhasil diimplementasikan di dalam sistem, diantaranya adalah:

1. *Tracker*

- Menjadi *server*.
- *Tracker* dapat menerima koneksi dari komponen lain.
- Mengelola ip *manager* dan *relay* yang terhubung dengan sistem.

2. *Manager*

- Menjadi client dan melakukan koneksi kepada *tracker*.
- Menjadi *server*.
- Mampu menerima koneksi dari komponen lain.

- Mengelola registrasi *user*.
- Mengelola *login user*.
- Mengelola jumlah *user* terkoneksi pada *relay*.
- Mengelola permintaan CRUD *roster*.
- Mengelola permintaan inisiasi *presence*.
- Mengelola permintaan *directed presence*.
- Mengelola permintaan pembaruan bio *user*.
- Mengelola permintaan *logout user*.

3. *Relay*

- Menjadi client dan melakukan koneksi kepada komponen *tracker* dan *manager*.
- Menjadi *server*.
- Mampu menerima koneksi dari komponen lain.
- Mengelola informasi *user* pada dirinya dan *relay* lain.
- Menerima pesan dari *user* dan *relay*.
- Mengirimkan pesan kepada *user* dan *relay*.

Berdasarkan keberhasilan pengujian *unit testing* fungsionalitas yang dapat dilihat pada setiap implementasi komponen, dapat dikatakan bahwa semua fungsionalitas dapat digunakan dan sesuai dengan tujuan aktivitas fungsionalitasnya. Hal ini menunjukkan bahwa dengan penggunaan arsitektur *microservice* tidak akan mengganggu fungsionalitas dari sistem yang dibuat, dan akan memberikan kemampuan baru dalam pengembangan dan penerapannya, seperti penggunaan bahasa pemrograman yang berbeda untuk setiap komponennya atau menambah jumlah komponen tertentu sesuai kebutuhan.

4. 10 Komunikasi Antar Aplikasi Dalam Sistem

Jika aplikasi dapat menerapkan tiga hal, yaitu melakukan koneksi dengan sistem, mengirimkan data kepada sistem, dan menerima data dari sistem. Maka aplikasi tersebut dapat melakukan komunikasi, baik itu aplikasi yang sama ataupun berbeda aplikasi. Komunikasi ini dapat dicapai jika kedua aplikasi tersebut mengikuti standar pesan pada sistem.

Seperti pada <https://github.com/Jarsters/socket-xmpp-json> terdapat dua aplikasi *user*, yang mana salah satu dikembangkan dengan python dan yang lainnya node.js. Kedua aplikasi ini dapat menerapkan tiga hal yang disebutkan, dan kedua aplikasi yang berbeda ini dapat saling berkomunikasi melalui *middleware* komunikasi yang sudah dibuat.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan implementasi dan pengujian untuk *middleware* komunikasi menggunakan XMPP dan arsitektur *microservice* ini, dapat disimpulkan bahwa:

1. *Socket programming* hanyalah saluran untuk pertukaran data, pertukaran data dapat dilakukan seperti apa saja. Jika tidak ada format acuan, sistem tidak akan bisa digunakan karena tidak memiliki format data yang jelas. Dibutuhkan sebuah protokol pertukaran data untuk menanganinya seperti XMPP.
2. Pemahaman tentang protokol yang digunakan sebagai landasan format pertukaran data harus dikuasai dengan benar. Pemahaman ini akan membantu jika pengembang ingin melakukan inovasi pada protokol tersebut.
3. Pembuatan *middleware* komunikasi menggunakan XMPP dan arsitektur *microservice* mempunyai tujuan khusus untuk menambahkan kemampuan meningkatkan kebutuhan yang dibutuhkan saja. Hal ini dapat dilihat dari pengimplementasian sistem, komponen yang berfungsi sebagai jalur komunikasi yaitu *relay* dapat digandakan. Tidak seperti *monolith* yang tidak memiliki kemampuan meningkatkan kebutuhan tertentu, *monolith* ketika ingin meningkatkan kebutuhan, wajib meningkatkan seluruh proses.
4. Pengubahan dari *monolith* ke *microservice* memberikan kemampuan untuk memberikan hasil yang terbaik dari setiap komponen. Hasil ini didapat karena setiap komponen dapat dikembangkan sesuai *best practice* kasus yang dihadapi, tanpa takut mengganggu proses komponen lain.

5.2 Saran

Adapun saran untuk penelitian selanjutnya adalah:

1. Keamanan jaringan pada penelitian ini belum diimplementasikan sama sekali, maka ada baiknya jika memperbaiki permasalahan keamanan jaringan yang ada. Mengamankan jaringan dengan algoritma enkripsi ECC dan mengamankan pengiriman data yang dilakukan menggunakan enkripsi AES.
2. Pada penelitian ini, fitur konfirmasi penambahan *roster* masih otomatis dilakukan oleh *server*. Bisa menambahkan spesifikasi lebih lanjut pada fitur ini.
3. Fitur untuk mengirimkan gambar atau pesan suara juga bisa ditambahkan.
4. XMPP memiliki banyak sekali catatan fitur, gunakan yang ingin diimplementasikan.

DAFTAR PUSTAKA

- Amira, K. (2021). *Pengertian Jaringan Komputer: Jenis-Jenis, Cara Kerja, dan Manfaat*. Gramedia. <https://www.gramedia.com/literasi/pengertian-jaringan-komputer>
- Ben Gorman. (2023). *TCP vs UDP: What's the Difference and Which Protocol Is Better?* Gen Digital Inc. <https://www.avast.com/c-tcp-vs-udp-difference>
- Biznet News. (2023a). *5 Keunggulan dan Fungsi VPS, Apa Saja?* PT Biznet Gio Nusantara. <https://www.biznetgio.com/news/5-keunggulan-dan-fungsi-vps>
- Biznet News. (2023b). *Perbedaan Antara Public IP dan Private IP*. PT Biznet Gio Nusantara. <https://www.biznetgio.com/news/perbedaan-public-ip-dan-private-ip>
- Breje, A. R., Gyorödi, R., Gyorödi, C., Zmaranda, D., & Pecherle, G. (2018). Comparative study of data sending methods for XML and JSON models. *International Journal of Advanced Computer Science and Applications*, 9(12), 198–204. <https://doi.org/10.14569/IJACSA.2018.091229>
- Chandler Harris. (2024). *Microservices vs. monolithic architecture*. <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>
- Faiz. (2023). *Ketahui Perbedaan IP Private dan IP Public*. Telkom University. <https://dte.telkomuniversity.ac.id/ketahui-perbedaan-ip-private-dan-ip-public/>
- Faradilla A. (2023). *Apa Itu IP Address? Pengertian, Jenis, dan Fungsinya*. Hostinger.Co.Id. <https://www.hostinger.co.id/tutorial/apa-itu-ip-address>
- Gupta, D., Shivankar, J., & Gugulothu, S. (2021). Instant messaging using xmpp. *Journal of Physics: Conference Series*, 1913(1), 8–12. <https://doi.org/10.1088/1742-6596/1913/1/012126>
- Maxwin, A. (2021). *TCP/IP concurrent connections*. <https://learn.microsoft.com/en-us/answers/questions/482793/tcp-ip-concurrent-connections>

- Oikarinen, J., & Reed, D. (1993). Internet Relay Chat Protocol. *Internet Engineering Task Force*, May, 1–65. <https://www.rfc-editor.org/info/rfc1459>
- P. Saint-Andre. (2011). *Extensible Messaging and Presence Protocol (XMPP): Core*. <https://xmpp.org/rfcs/rfc6120.html>
- Python. (2024). *About Python*. Python Software Foundation. <https://www.python.org/about/>
- Rekhter, Y., Karrenberg, D., J., de G. G., Lear, E., & Moskowitz, R. G. (1996). *Request for Comments: 1918 - Address Allocation for Private Internets*. 1–9.
- Revou. (2024). *Middleware*. PT Revolusi Cita Edukasi. <https://revou.co/kosakata/middleware>
- Roberts, P. A., & Challinor, S. (2000). IP address management. *BT Technology Journal*, 18(3), 127–136. <https://doi.org/10.1023/A:1026749131441>
- Saint-Andre, P., Smith, K., & Tronçon, R. (2009). *XMPP : The Definitive Guide*.
- Shafiei, B. M., Idiranmanesh, F., & Iranmanesh, F. (2012). Socket programming. *Advances in Environmental Biology*, 6(5), 1812–1822. <https://doi.org/10.1016/b978-155860826-9/50016-1>
- Verma, K. (2023). *Socket Programming in Python*. GeeksforGeeks. <https://www.geeksforgeeks.org/socket-programming-python/>
- Virga, J. (2020). *UDP Communication*. MoinMoin Wiki Engine. <https://wiki.python.org/moin/UdpCommunication>